UNIVERSITY OF THE PEOPLE

# Designing and Evaluating an Integrated Access-Control Framework for Electronic Health Records in a Saudi Government Hospital

**A Capstone Project Report of MSIT 5910**

*Submitted by:*
**Taghreed Y. Almirabi (C1958065)**

*For the partial fulfilment of the requirements for the degree of*
**Master of Science in Information Technology**

*Supervised by:*
**Dr. Stella T. Whyte**

**Department of CS & MSIT**
University of the People, Pasadena, CA 91101, United States

**AY2027-T1, October 2026**

# ABSTRACT

Using the design science research methodology, this capstone designed, implemented, and evaluated an integrated access-control framework for electronic health records (EHRs) in a modeled Saudi government hospital. The project addresses fragmented enforcement across role permissions, authentication, clinical context, emergency access, and audit review. A Python prototype combines six-role role-based access control, patient-assignment and duty context, step-up multifactor authentication, controlled break-glass authorization, HMAC-signed audit events, and SQLite persistence behind one application service. Only synthetic identifiers were used, and no compliance claim is made. Evaluation included 25 automated tests, 36 labeled decision cases, a documented 300-request local benchmark, a five-criterion clarity heuristic, and reviews by two healthcare IT experts. A system test exposed a whitespace-only user-identifier defect; after correction, all 25 tests passed on release v1.3.0-final (commit 0bb3017). The labeled cases achieved 100% accuracy with no false permits. Mean latency was 3.196 ms, p99 latency was 4.205 ms, throughput was 312.18 requests per second, and peak incremental memory was 0.127 MB. Interface clarity scored 4.60/5, while the experts rated the framework 4.85/5 and recommended a controlled pilot. These results demonstrate coherent, traceable behavior within the prototype boundary, not production security, clinical usability, or hospital-scale capacity. A Docker deployment plan and maintenance plan define the next controlled stage.

*Keywords:* electronic health records, access control, multifactor authentication, break-glass access, audit integrity, Saudi Arabia

# ACKNOWLEDGEMENT

I thank my instructor, Dr. Stella T. Whyte, for the detailed feedback that strengthened the project's success criteria, testing boundaries, benchmark documentation, release traceability, and APA 7 presentation. I also thank the Department Chair and faculty of Computer Science and MSIT, the University of the People, and the two healthcare IT experts who reviewed the prototype. Finally, I am grateful to my family for their patience and support throughout the proposal, design, implementation, testing, and reporting stages.

UNIVERSITY OF THE PEOPLE

# LIST OF TABLES

Table 1. Stakeholders, Concerns, and Expected Value....13

Table 2. Research Questions, Objectives, and Success Criteria....15

Table 3. Comparative Analysis of Existing Access-Control Approaches....21

Table 4. Implemented Role–Permission Matrix....30

Table 5. Integrated Module Inputs and Outputs....31

Table 6. Audit Event Data Model (audit_events Table)....33

Table 7. Functional and Non-Functional Requirements and Acceptance Measures....34

Table 8. Unit 7 System, Acceptance, and Stability Test Cases....40

Table 9. Integrated Evaluation Results....45

Table 10. Expert Reviewer Ratings on the Ten Validation Statements....47

Table 11. Requirements Traceability Matrix....49

Table 12. Objective Status Against Success Criteria....50

Table 13. Proposed Maintenance and Monitoring Schedule....58

Table 14. Post-Deployment Risks and Controls....59

UNIVERSITY OF THE PEOPLE

# LIST OF FIGURES

Figure 1. Integrated EHR Access-Control Architecture (Logical Design).....24

Figure 2. Implemented Prototype Architecture and Module Communication.....25

Figure 3. Controlled Break-Glass Workflow.....32

Figure 4. End-to-End Demonstration and Persisted Audit Summary.....39

Figure 5. Initial Failed Blank-Identifier System Test.....41

Figure 6. Corrected Validation and Focused Passing Test.....42

Figure 7. Complete 25-Test Regression Result.....43

Figure 8. Integrated System Evaluation Chart.....46

Figure 9. Verified Unit 7 GitHub Release.....51

Figure 10. Verified Final Release v1.3.0-final.....52

Figure D1. Reviewer 1 Evaluation Form, Page 1 (Information and Review Steps).....70

Figure D2. Reviewer 1 Evaluation Form, Page 2 (Ratings).....71

Figure D3. Reviewer 1 Evaluation Form, Page 3 (Open Feedback and Declaration).....72

Figure D4. Reviewer 2 Evaluation Form, Page 1 (Information and Review Steps).....73

Figure D5. Reviewer 2 Evaluation Form, Page 2 (Ratings).....74

Figure D6. Reviewer 2 Evaluation Form, Page 3 (Open Feedback and Declaration).....75

# SYMBOLS AND ABBREVIATIONS

| | |
|---|---|
| **ABAC** | Attribute-Based Access Control |
| **API** | Application Programming Interface |
| **CI/CD** | Continuous Integration and Continuous Delivery/Deployment |
| **DSRM** | Design Science Research Methodology |
| **ECC** | Essential Cybersecurity Controls |
| **EHR** | Electronic Health Record |
| **FR** | Functional Requirement |
| **HIM** | Health Information Management |
| **HMAC** | Hash-Based Message Authentication Code |
| **HR** | Human Resources |
| **IAM** | Identity and Access Management |
| **ISO** | International Organization for Standardization |
| **IT** | Information Technology |
| **JSON** | JavaScript Object Notation |
| **MFA** | Multifactor Authentication |
| **NCA** | National Cybersecurity Authority (Saudi Arabia) |
| **NFR** | Non-Functional Requirement |
| **NIST** | National Institute of Standards and Technology |
| **OWASP** | Open Worldwide Application Security Project |
| **PDP** | Policy Decision Point |

| | |
|---|---|
| **PDPL** | Personal Data Protection Law (Saudi Arabia) |
| **PEP** | Policy Enforcement Point |
| **RBAC** | Role-Based Access Control |
| **RQ** | Research Question |
| **SDAIA** | Saudi Data and Artificial Intelligence Authority |
| **SIEM** | Security Information and Event Management |
| **SOC** | Security Operations Center |
| **SQL** | Structured Query Language |
| **SSDF** | Secure Software Development Framework |
| **SSO** | Single Sign-On |
| **TLS** | Transport Layer Security |

UNIVERSITY OF THE PEOPLE

# CONTENTS


ABSTRACT....2

ACKNOWLEDGEMENT....3

LIST OF TABLES....4

LIST OF FIGURES....5

SYMBOLS AND ABBREVIATIONS....6

CONTENTS....8

CHAPTER 1: INTRODUCTION....11

1.1 Background of the Study....11

1.2 Problem Statement....12

1.3 Aim and Objectives of the Study....13

1.4 Scope and Limitations....15

1.5 Significance of the Project....16

1.6 Report Organization....16

CHAPTER 2: LITERATURE REVIEW....17

2.1 Overview of Related Work....17

2.2 Comparative Analysis of Existing Approaches....20

2.3 Research Gap Identification....21

CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY ..... 23

3.1 System Architecture ..... 23

3.2 Methodologies and Tools Used ..... 25

3.3 Data Flow and System Components ..... 29

3.4 Design Diagrams ..... 32

3.5 Functional and Non-Functional Requirements ..... 33

3.6 Ethical and Security Considerations ..... 34

CHAPTER 4: RESULTS AND EVALUATION ..... 36

4.1 Implementation Overview ..... 36

4.2 Output and Screenshots ..... 38

4.3 Evaluation Metrics ..... 44

4.4 Deployment and Version-Control Evidence ..... 50

CHAPTER 5: DISCUSSION ..... 53

5.1 Interpretation of Results ..... 53

5.2 Comparison with Literature Findings ..... 54

5.3 Implications and Limitations ..... 55

5.4 Maintenance and Operational Sustainability ..... 57

5.5 Post-Deployment Risks and Mitigation ..... 58

5.6 Overall Interpretation ..... 59

CHAPTER 6: CONCLUSION AND FUTURE WORK ..... 60

6.1 Summary of Main Findings....60

6.2 Implications and Project Contribution....60

6.3 Lessons Learned and Challenges....61

6.4 Future Work....61

6.5 Conclusion....62

REFERENCES....63

APPENDICES....67

Appendix A: Test Commands and Evidence Map....67

Appendix B: Configuration and Deployment Summary....67

Appendix C: Repository and Release....68

Appendix D: Expert Reviewer Validation Instrument and Completed Forms....69

Appendix E: Code Excerpts....76

# CHAPTER 1: INTRODUCTION

## 1.1 Background of the Study

Electronic health records (EHRs) make longitudinal clinical information available across departments and can improve continuity, coordination, and decision making. The same concentration of diagnoses, medications, allergies, laboratory results, and personal identifiers creates a high-value target and a serious duty of confidentiality. Access control in healthcare therefore cannot be reduced to keeping outsiders away: it must ensure that legitimate employees receive only the actions and patient context needed for their work, that urgent care is not blocked, and that exceptional access can be explained afterward.

In a large hospital those decisions draw on several systems: human resources, identity and access management (IAM), the authentication service, the EHR, care-assignment systems, and monitoring. When they are evaluated independently, a person may keep an old department privilege, an administrator may inherit unnecessary clinical access, or an emergency override may be available without review. This capstone addresses that coordination problem through one explainable decision flow.

The Saudi context makes the topic especially relevant. Health information is sensitive personal data whose access should be limited to the personnel and purposes necessary for health services; the Saudi Personal Data Protection Law (PDPL) provides the privacy foundation, while the National Cybersecurity Authority's (NCA) Essential Cybersecurity Controls address identity and access management, event logging, configuration, and resilience (NCA, 2024; Saudi Data and Artificial Intelligence Authority [SDAIA], 2023). This report uses those sources as design

guidance and does not present the prototype as certified, compliant, or approved for a government hospital.

## 1.2 Problem Statement

The project models a large Saudi government hospital whose EHR users change wards, shifts, and duties. It uses six roles and synthetic records rather than inspecting a real deployment. The central problem is fragmented control: directories, authentication services, EHR permissions, and monitoring tools may evaluate different parts of the same request. Consequently, transferred staff may retain old privileges, valid roles may bypass shift or patient-assignment checks, uniform MFA may burden routine work while leaving exports underprotected, emergency access may be too slow or broad, and separate logs may obscure who accessed a record and why. These are modeled design risks, not claims about a named hospital.

Evidence supports this problem context. A systematic review of 20 EHR access-control studies found that attribute-based authorization was common, but implementation, standards compliance, MFA, emergency access, patient consent, and accountability were frequently incomplete (Cobrado et al., 2024). A review of 11 Saudi studies reported uneven safeguards, policy breaches, and training gaps, concluding that confidentiality depends on both technical and organizational practice (Almaghrabi & Bugis, 2022). Neither review supplies a tested flow that connects identity assurance, roles, runtime context, emergency exceptions, and audit review for the modeled setting.

The modeled hospital therefore lacks a unified, testable method for coordinating identity assurance, role permissions, runtime context, emergency exceptions, and audit review. This fragmentation can produce excessive access, inappropriate denial, weak accountability, and

inconsistent evidence. The project addresses the problem by designing and evaluating a framework that makes each EHR decision limited, explainable, and traceable while preserving rapid access in legitimate emergencies. Table 1 summarizes the affected stakeholders and expected value.

**Table 1**

*Stakeholders, Concerns, and Expected Value*

| Stakeholder | Primary concern | Expected value from the framework |
|---|---|---|
| Patients | Confidentiality, accurate records, timely treatment | Fewer unnecessary disclosures and a safe emergency pathway |
| Clinical staff | Fast access, understandable denials, accountability | Permissions that follow duties without repeated manual requests |
| HIM and privacy teams | Purpose limitation, disclosure control, evidence | Consistent rules and reviewable audit records |
| IT, IAM, and SOC teams | Lifecycle events, privileged access, alert quality | Defined ownership, structured events, fewer orphaned rights |
| Hospital leadership | Patient safety, continuity, compliance risk | Measurable control coverage and a phased roadmap |

*Note.* HIM = health information management; IT = information technology; IAM = identity and access management; SOC = security operations center.

## 1.3 Aim and Objectives of the Study

The aim of this study is to design, implement, and evaluate an integrated access-control framework for EHRs in a modeled Saudi government hospital that coordinates identity assurance, role permissions, clinical context, emergency access, and tamper-evident audit evidence within one explainable and testable decision flow. The aim is pursued through three research questions and five measurable objectives, following the design science research methodology defined in Section 3.2.1.

Three research questions guide the study. RQ1 asks how stable role permissions and changing clinical context can be combined without creating an opaque policy model. RQ2 asks whether a lightweight prototype can enforce normal, high-risk, and emergency decisions while producing complete and tamper-evident audit evidence. RQ3 asks what functional tests, local performance measurements, and structured maintenance planning reveal about the prototype's readiness for a controlled next stage.

Five objectives operationalize the questions: O1 maps six hospital roles to the minimum necessary EHR actions; O2 defines authentication, authorization, context, emergency, and audit requirements with testable acceptance measures; O3 implements a rules-based Python prototype that evaluates synthetic requests and persists signed audit evidence; O4 tests normal, denied, high-risk, emergency, and misuse scenarios at unit, integration, system, and acceptance levels and measures local performance; and O5 obtains structured feedback on clarity and practicality through a five-criterion developer heuristic and a structured validation by two healthcare IT experts (Section 4.3.3; Appendix D). Table 2 connects each objective to its research question, its success criterion, and the chapter in which the evidence is reported.

**Table 2**

*Research Questions, Objectives, and Success Criteria*

| RQ | Objective | Success criterion | Evidence |
|---|---|---|---|
| RQ1 | O1: Six-role least-privilege matrix | All prohibited role–action pairs denied in the automated test set (100%) | Sections 3.3, 4.2, 4.3 |
| RQ1 | O2: Testable requirements | Every functional requirement linked to at least one automated test and evidence item (100% traceability) | Sections 3.5, 4.3 |
| RQ2 | O3: Executable prototype with signed, persisted audit evidence | One audit event per evaluated request; altered events detected; break-glass decisions persisted | Sections 4.1, 4.2 |
| RQ2 | O4: Multi-level testing and local benchmark | ≥95% labeled-decision accuracy with zero false permits; all automated tests passing on the released commit; benchmark conditions, hardware, and latency percentiles documented | Sections 4.2, 4.3 |
| RQ3 | O5: Structured usability feedback | Heuristic clarity ≥4.0/5; two-reviewer mean ≥4.0/5 with no unresolved critical concern | Sections 4.3, 5.3; Appendix D |
| RQ3 | Deployment and maintenance readiness | Verified release tag linked to tested commit; documented deployment configuration and maintenance plan | Sections 4.4, 5.4, 5.5 |

*Note.* RQ = research question; O = objective; ≥ = greater than or equal to. Section and appendix references indicate the location of supporting evidence within this report.

## 1.4 Scope and Limitations

The completed scope covers six roles, baseline permissions, authentication and account checks, patient assignment and duty context, network and device context, step-up MFA, patient-specific break-glass access, HMAC-protected audit events, SQLite persistence, automated tests, performance measurement, deployment files, Git traceability, and maintenance planning. The logical design also includes joiner–mover–leaver and SIEM functions; the prototype represents them through account-state flags and a queryable audit store rather than live HR or SIEM integration. The original spreadsheet concept was replaced with Python so the artifact could be tested, integrated, benchmarked, and versioned. Exclusions are live EHR integration, real

credentials or health data, certification, production availability, clinical safety validation, penetration testing, and hospital-scale concurrency. The result is an evaluated design artifact, not a clinical product.

## 1.5 Significance of the Project

The project's practical contribution is integration. RBAC, contextual authorization, MFA, emergency access, audit integrity, testing, and maintenance are often discussed as separate safeguards; the prototype shows how one request can move through those concerns in an ordered and reviewable way. For a hospital team, the resulting artifacts—architecture, rules, tests, metrics, risk register, configuration plan, and release history—can support stakeholder discussion before expensive vendor or infrastructure decisions. For academic work, the project demonstrates how security claims can be tied to explicit evidence and limitations.

## 1.6 Report Organization

Chapter 2 reviews the literature and identifies the research gap. Chapter 3 explains the methodology, architecture, algorithm, requirements, and evaluation plan. Chapter 4 reports implementation, testing, metrics, expert review, and traceability. Chapter 5 interprets the results and addresses limitations, deployment, maintenance, and risk. Chapter 6 summarizes the contribution and future work. The appendices provide commands, configuration, release evidence, signed reviewer forms, and code excerpts.

# CHAPTER 2: LITERATURE REVIEW

## 2.1 Overview of Related Work

### *2.1.1 EHR Confidentiality and the Saudi Context*

EHRs improve continuity of care by making diagnoses, medications, allergies, laboratory results, and prior encounters available across departments. Concentrating those data also creates privacy and safety obligations. A review of 49 studies found RBAC to be the preferred model, audit logging in about half of proposals, emergency override in roughly one third, and limited attention to training; it called for stronger standards adoption and deployment evidence (Fernández-Alemán et al., 2013). A Saudi review likewise identified confidentiality risks and showed that safeguards, staff awareness, and organizational practice must work together (Almaghrabi & Bugis, 2022). This project therefore treats fragmented access governance as a design problem that can be studied without exposing a real organization.

The PDPL gives the project a local privacy foundation: health information is sensitive personal data, and access and processing must be limited to the smallest number of personnel and to what is necessary for health services (SDAIA, 2023). ECC-2:2024 provides the complementary security perspective, addressing governance, identity and access management, system protection, event logging, and resilience for national entities (NCA, 2024). The framework is described as aligned with these sources, not certified as compliant.

### *2.1.2 Access-Control Models for Clinical Work*

A hospital can apply several access-control models. Discretionary access control lets an owner grant permissions, which is hard to govern for shared clinical records; mandatory access

control uses central classifications and is strong but inflexible. RBAC, formalized in the NIST standard model, assigns permissions to roles such as physician, nurse, pharmacist, laboratory technician, HIM employee, and IT administrator, so that administrators change a role assignment instead of many individual permissions when a person changes position (Ferraiolo et al., 2001).

Static RBAC cannot account for a permitted physician who is off duty, unassigned to the patient, or using an unmanaged device. ABAC can express these conditions, but large attribute sets are harder to administer and audit (Hu et al., 2014). Earlier EHR literature favored RBAC (Fernández-Alemán et al., 2013), whereas a recent review found ABAC most frequently proposed and identified missing MFA, emergency access, and accountability as recurring gaps (Cobrado et al., 2024). Hospital audit research also shows that legitimate care sometimes exceeds static assignments, requiring governed exceptions rather than rigid denial (Røstad & Edsberg, 2006). The project therefore uses context-aware RBAC: roles establish an auditable baseline, and a small context set narrows access at request time.

### *2.1.3 Zero Trust, Authentication, and Emergency Access*

Zero trust rejects network location as sufficient proof of trust. NIST SP 800-207 separates policy decision and enforcement and calls for explicit authorization using identity, resource, and context (Rose et al., 2020). Healthcare's mobile staff, shared workstations, connected devices, and ransomware exposure make this approach especially relevant (Vukotich, 2023). In the design, the policy enforcement point (PEP) intercepts each request, and the policy decision point (PDP) evaluates role and context before the EHR action proceeds.

Authentication must resist compromise without delaying care. NIST SP 800-63B-4 defines assurance levels, session controls, step-up authentication, and phishing-resistant options (Temoshok et al., 2025). Healthcare MFA research shows that usability, device constraints, and

deployment context matter (Suleski et al., 2023), while adaptive MFA can use user, environment, device, and use-case data to raise assurance selectively (Suleski & Ahmed, 2023). The design therefore retains routine authenticated sessions but requires step-up MFA for remote access, unmanaged devices, privileged changes, and record export.

Emergency access is the most sensitive exception. A strict denial can delay care when a patient arrives unconscious, while an unrestricted override can become a route for inappropriate access. Brucker and Petritsch (2009) formalized break-glass as a controlled extension of an access-control model in which exceptional permissions are explicitly granted, logged, and reviewed rather than hidden in administrator overrides, and Cobrado et al. (2024) list the absence of emergency access and accountability features among the most prominent gaps in EHR access-control proposals. The project's break-glass mechanism therefore requires an authenticated clinician, a named patient, a structured justification, a short expiry, immediate logging, and retrospective review; it bypasses selected contextual conditions but never identity verification or auditing.

### *2.1.4 Audit Integrity, Requirements, and Traceability*

Audit evidence must be complete and protected. NIST recommends controlled retention, protection from unauthorized change, and regular review (Kent & Souppaya, 2006), while the OWASP Top 10 recommends server-side, deny-by-default enforcement and logging of access-control failures (OWASP Foundation, 2025). These principles shaped the ordered guards, signed events, and logging of denials and MFA challenges. ISO/IEC/IEEE 29148:2018 distinguishes functional requirements from quality requirements and requires them to be unambiguous and verifiable (International Organization for Standardization [ISO], 2018). Linking each

requirement to the objective and evidence makes claims such as prohibited role–action pairs being denied testable (Summers, 2020).

#### *2.1.5 System Quality, Testing, Secure Development, and Maintenance*

ISO/IEC 25010:2023 defines nine complementary product-quality characteristics, showing why accuracy alone is not sufficient evidence (ISO, 2023). Accordingly, the evaluation combines functional tests, persistence checks, latency, interface clarity, configuration controls, and maintenance planning. ISO/IEC/IEEE 29119-1:2022 separates unit, integration, system, and acceptance testing, each answering a different question about the product (ISO, 2022b). The NIST Secure Software Development Framework integrates secure practices and retained evidence across the life cycle (Souppaya et al., 2022), while ISO/IEC/IEEE 14764:2022 defines corrective, adaptive, perfective, and preventive maintenance (ISO, 2022a).

## 2.2 Comparative Analysis of Existing Approaches

Table 3 compares the main approaches in the literature against the needs of the modeled hospital. None of them alone combines stable roles, clinical context, risk-adaptive authentication, governed emergencies, and tamper-evident evidence in one executable, tested design, which is the combination the prototype implements.

**Table 3**

*Comparative Analysis of Existing Access-Control Approaches*

| Approach and source | Strengths | Limitations in a hospital | Relevance to this project |
|---|---|---|---|
| RBAC (Ferraiolo et al., 2001) | Administrable; maps to job roles | Static; ignores shift, device, care relationship | Stable permission baseline for six roles |
| ABAC (Hu et al., 2014; Cobrado et al., 2024) | Expresses rich context | Policies grow complex and hard to audit | Small fixed context set added to RBAC |
| Break-glass (Brucker & Petritsch, 2009) | Preserves availability with explicit exceptions | Needs justification, expiry, and review | Separate, time-limited, always-audited route |
| Adaptive MFA (Suleski & Ahmed, 2023; Temoshok et al., 2025) | Matches assurance to risk | Needs identity-provider integration | STEP_UP_MFA decision for high-risk context |
| Zero trust (Rose et al., 2020; Vukotich, 2023) | Per-request authorization; PDP/PEP separation | Needs device posture and continuous signals | PDP/PEP separation and per-request evaluation |
| Audit-trail analysis (Røstad & Edsberg, 2006; Kent & Souppaya, 2006) | Reveals real needs; supports accountability | Reactive; logs must be protected | HMAC-signed events for every decision, including denials |

*Note.* RBAC = role-based access control; ABAC = attribute-based access control; MFA = multifactor authentication; PDP = policy decision point; PEP = policy enforcement point; HMAC = keyed-hash message authentication code. STEP_UP_MFA denotes a decision requiring additional authentication for a high-risk request.

## 2.3 Research Gap Identification

The sources provide necessary but separate pieces: Saudi privacy and cybersecurity expectations, access-control models, identity assurance, emergency access, logging, secure development, testing, and maintenance. The practical gap is their connection—how identity status reaches a decision, when MFA should increase, how a care relationship narrows a role,

how emergency access remains accountable, which audit fields are required, and how success is measured.

This capstone contributes a blueprint, executable prototype, and evaluation method rather than a new cryptographic algorithm. Each problem maps to a module, each requirement to evidence, and each success claim to a threshold. The design combines RBAC, a limited context set, risk-triggered MFA, explicit break-glass access, signed events, layered tests, synthetic data, versioned releases, and maintenance controls (ISO, 2022a, 2022b, 2023; Souppaya et al., 2022). Successful synthetic evaluation demonstrates internal consistency and clarity, not production security or legal compliance.

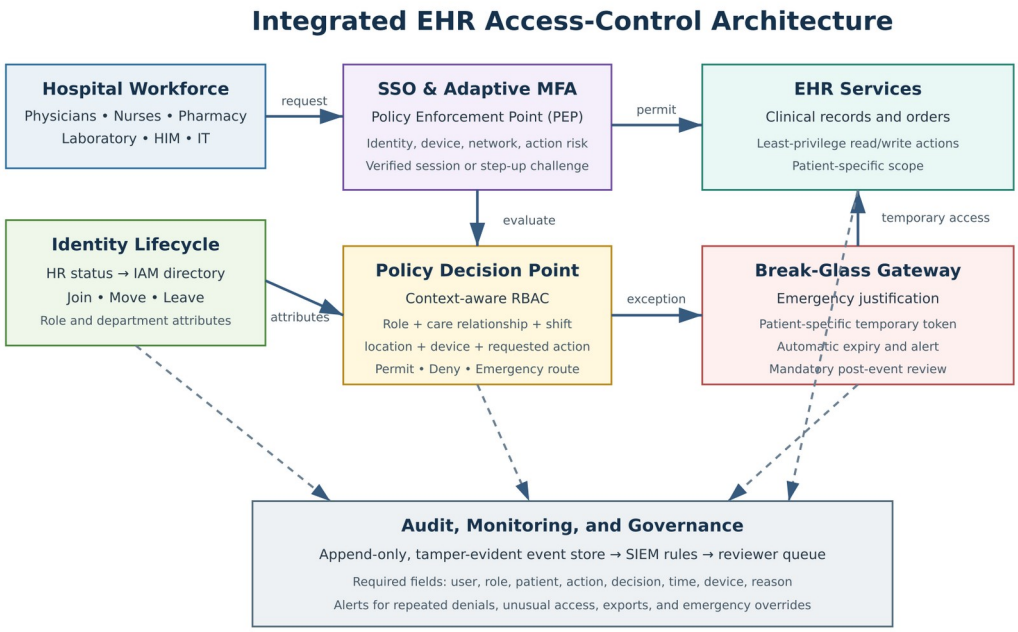
# CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY

## 3.1 System Architecture

Figure 1 presents the logical architecture. The control plane contains identity information, authentication risk rules, the RBAC matrix, contextual policies, and emergency-access rules; the data plane contains the PEP and the EHR services that execute an approved action. Separating these responsibilities keeps policy logic centralized, ensures server-side enforcement, and provides one location for decision evidence. Identity changes originate in human resources and flow to IAM, and every module produces structured evidence for the security information and event management (SIEM) service.

**Figure 1**

*Integrated EHR Access-Control Architecture (Logical Design)*



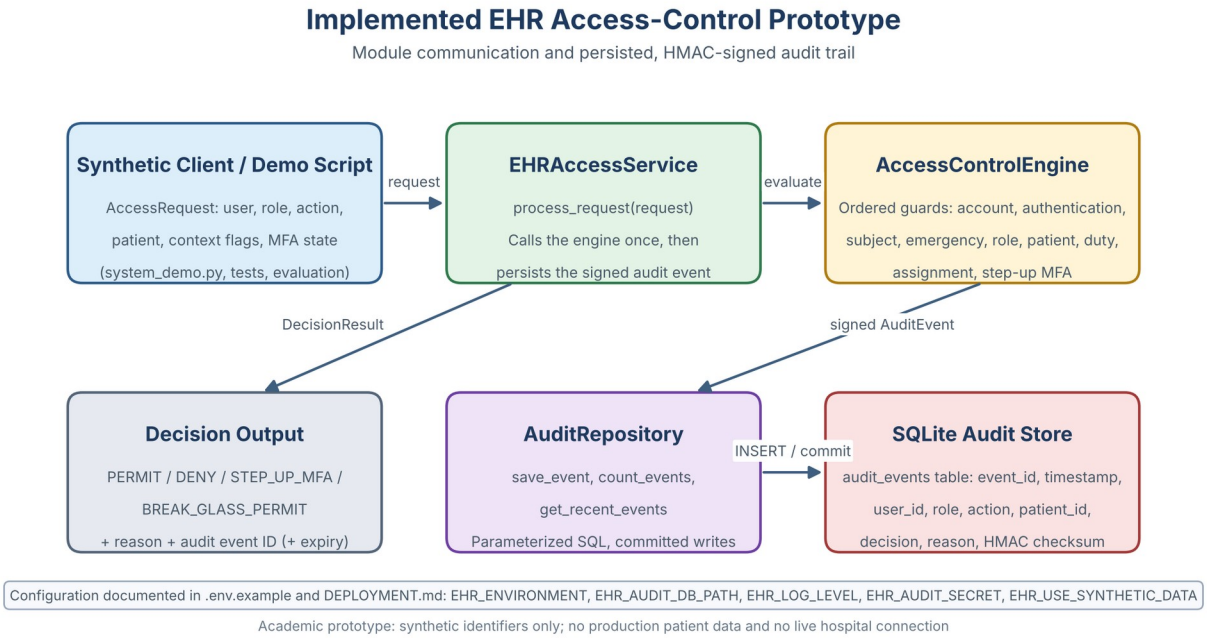

*Note.* Solid arrows show the authorization path; dashed arrows show security events sent to monitoring and governance. The design connects workforce identity, adaptive authentication, policy decisions, EHR enforcement, emergency access, and governance evidence.

A normal transaction follows five steps: the PEP receives the identity and EHR action; authentication validates the session and any step-up requirement; the PDP retrieves the active role and baseline permissions; contextual rules check the care relationship, shift, location, device, patient, and operation; and the PEP forwards a permitted operation or returns an explained denial. A shared identifier links the evidence. In the prototype (Figure 2), a client creates an AccessRequest, EHRAccessService invokes AccessControlEngine, and AuditRepository saves the signed AuditEvent in SQLite before the DecisionResult returns. The service and engine

represent the PEP/PDP boundary, and the queryable audit store represents the prototype's monitoring evidence.

**Figure 2**

*Implemented Prototype Architecture and Module Communication*




*Note.* The service coordinates authorization and persistence while the engine and repository remain independently testable. Decision labels and configuration names match the released source code.

## 3.2 Methodologies and Tools Used

### *3.2.1 Research and Development Methodology*

The project followed the design science research methodology (DSRM), in which constructing and evaluating a useful artifact is the research contribution (Hevner et al., 2004; Peffers et al., 2007). Its six activities map to this study: problem identification (Chapters 1–2), solution objectives (Table 2), design and development (Chapter 3 and Section 4.1),

demonstration (Section 4.2), evaluation (Section 4.3), and communication through the report, repository, and portfolio. Hevner et al.'s (2004) guidelines served as a design-science checklist. The study is therefore applied and artifact-centered: it asks whether the implemented design meets predefined objectives and reports both evidence and limitations rather than testing a population hypothesis.

Development was iterative and incremental. Units 1–3 defined the problem, proposal, architecture, requirements, evaluation plan, and repository; Unit 4 addressed CI/CD; Unit 5 implemented and unit-tested the engine; Unit 6 integrated persistence, evaluation, and deployment artifacts; Unit 7 added system and acceptance tests, defect correction, and maintenance planning; and the final stage added the 36-case benchmark and expert validation. Commits, merges, and tags connect Chapter 4 results to repository states.

### *3.2.2 Core Decision Algorithm*

The algorithm uses ordered guards so a later convenience rule cannot bypass an earlier safety condition. It first checks account state, authentication, and a nonblank user identifier. Emergency requests then enter a constrained break-glass path that still requires an eligible clinical role, patient, clinical action, and justification. Normal requests must satisfy role permission, patient identification, duty status, and clinician–patient assignment. Record export, account management, off-network access, or an unmanaged device returns STEP_UP_MFA unless MFA is already verified. Every path ends in the same method, which creates an explained decision and signed audit event.

```
validate: account active, session authenticated, user identifier present
if emergency request:
    require clinical role (physician or nurse), patient identifier,
    clinical action, and a justification of at least ten characters
    grant BREAK_GLASS_PERMIT with an expiry timestamp
else:
    require role permission for the requested action
    require patient identifier for patient-level actions
    require on-duty status
    require clinician-patient assignment (physicians and nurses)
    if high-risk action, off-network, or unmanaged device
        and MFA not verified: return STEP_UP_MFA
    otherwise: return PERMIT
create reasoned decision and HMAC-signed audit event; persist; return
```

STEP_UP_MFA is a distinct decision so the application can request stronger authentication and resubmit, consistent with NIST step-up guidance (Temoshok et al., 2025). BREAK_GLASS_PERMIT includes an expiry timestamp. The logical design proposed 30 minutes, whereas the prototype uses a 15-minute constant to be replaced by approved hospital policy; both values are disclosed.

### *3.2.3 Audit Integrity Mechanism*

Each audit event records an identifier, timestamp, user, role, action, patient, decision, reason, and checksum. The engine serializes these fields as canonical JSON and signs them with HMAC-SHA256. Verification recomputes the value and uses hmac.compare_digest. This mechanism is tamper-evident, not immutable: it detects an altered row but cannot stop an attacker who controls both the signing key and database from replacing the row and checksum. Chapter 5 addresses that limitation.

### *3.2.4 Tools and Technologies*

Python 3.12 was selected for readability and its standard hmac, json, sqlite3, and tracemalloc modules. pytest runs the tests; SQLite provides lightweight transactional persistence; Matplotlib creates the evaluation chart; tracemalloc measures peak incremental memory; Docker documents a portable runtime; and Git/GitHub preserves branches, pull requests, tags, and

releases. The Unit 4 CI/CD plan uses GitHub Actions to test pull requests, block failed security checks, and retain evidence. Automatic hospital deployment remains excluded, consistent with secure-development guidance (Souppaya et al., 2022).

### *3.2.5 Evaluation and Testing Method*

Evaluation used synthetic requests with predefined expected decisions and four instruments. First, 25 automated tests covered unit, integration, system, and acceptance behavior; white-box tests inspected known rule branches and HMAC behavior, while black-box assertions compared requests with expected outcomes. Temporary databases isolated tests. Second, decision accuracy equaled matched outcomes divided by labeled cases. Unit 6 used four cases; the final benchmark used 36 cases (six per role) spanning permits, prohibited role–action pairs, missing context, high-risk escalation, and valid or invalid emergencies, and also counted false permits. Third, after ten warm-up requests, the integrated service processed 300 sequential physician requests through SQLite. time.perf_counter measured each request; the script reported mean, median, p95, and p99 latency, elapsed-time throughput, and peak incremental memory from tracemalloc. Fourth, a five-item developer heuristic rated decision labels, explanations, input consistency, audit traceability, and ease of learning. Predefined thresholds required at least 95% labeled accuracy with zero false permits, correct prohibited and high-risk outcomes, complete emergency evidence, one signed event per request, all tests passing on the release, mean latency below 250 ms, full requirement traceability, and usability of at least 4.0/5. Two healthcare IT experts then reviewed the demonstration with the instrument in Appendix D. Their feedback assesses practitioner-facing clarity and plausibility, not clinical usability in a live workflow.

## 3.3 Data Flow and System Components

### *3.3.1 Identity and Account-State Handling*

The logical identity-lifecycle module accepts approved HR events: joining creates a minimum role, moving removes old-department rights before adding new ones, leaving disables the account and sessions, and privileged changes require separate approval. The prototype represents these outcomes through account_active, authenticated, and on_duty attributes. It can therefore test inactive, unauthenticated, and off-duty denial without a real HR connection; automated joiner–mover–leaver processing remains future work.

### *3.3.2 Authentication and Policy Enforcement*

The PEP is the protected entry point. A managed device, hospital network, and routine action preserve the current session; remote access, an unmanaged device, or a high-risk action requires step-up MFA. The request carries on_hospital_network, managed_device, and mfa_verified flags, and the engine returns STEP_UP_MFA when higher-risk context lacks verified MFA. Phishing-resistant authenticators are preferred for remote or privileged activity.

### *3.3.3 Context-Aware Authorization and the Role–Permission Matrix*

The PDP checks the role–action permission before evaluating patient identifier, duty status, and, for physicians and nurses, an active patient assignment. Missing mandatory context causes denial; favorable context cannot grant an action absent from the role baseline. Every result includes a reason. Table 4 reproduces the matrix implemented in the released source.

**Table 4**

*Implemented Role–Permission Matrix*

| Role | Permitted actions | Boundary and context |
|---|---|---|
| Physician | view_patient_record; update_clinical_notes; prescribe_medication | Patient identifier and assignment required; eligible for break-glass |
| Nurse | view_patient_record; update_clinical_notes; administer_medication | Patient identifier and assignment required; eligible for break-glass |
| Pharmacist | view_patient_record; dispense_medication | Patient identifier required; no clinical-note editing |
| Laboratory technician | view_patient_record; record_lab_result | Patient identifier required; no unrelated chart actions |
| Health information manager | view_patient_record; export_patient_record; review_audit_logs | Export is high risk and requires verified MFA |
| IT systems administrator | manage_user_accounts; review_audit_logs | No clinical-record access by default; account management requires verified MFA |

*Note.* All patient-level actions require a patient identifier; all roles require an active account, an authenticated session, a valid user identifier, and on-duty status. Access from outside the hospital network or from an unmanaged device requires verified MFA for any role.

### *3.3.4 Emergency Break-Glass Module*

Break-glass is a governed exception, not a master account. An authenticated physician or nurse must identify the patient, request a clinical action, and provide a justification of at least ten characters. The engine returns a 15-minute BREAK_GLASS_PERMIT and signed event; nonclinical roles, missing patients, nonclinical actions, and empty justifications are denied. Inactivity timeout and explicit closure remain future work (Section 6.4). The logical workflow in Figure 3 adds immediate notification, retrospective review, closure or escalation, and evidence retention.

### *3.3.5 Audit, Monitoring, and Evidence*

Every evaluated request creates an event, while clinical narrative is excluded to reduce log privacy risk. The repository uses parameterized SQL, committed transactions, bounded retrieval, and reverse-time ordering. Proposed SIEM rules cover repeated denials, missing care relationships, unusual volume, export, privileged changes, terminated-user attempts, and repeated emergencies. An alert initiates investigation; it is not proof of misconduct. Table 5 summarizes component inputs and outputs.

**Table 5**

*Integrated Module Inputs and Outputs*

| Component | Primary responsibility | Input | Output |
|---|---|---|---|
| AccessControlEngine | RBAC, context, step-up MFA, break-glass, audit signing | AccessRequest | DecisionResult and AuditEvent |
| EHRAccessService | Coordinate authorization and persistence | Request and module references | Saved decision and retrieval methods |
| AuditRepository | Create schema; store and query events | AuditEvent or query limit | Persistent row, count, recent events |
| system_demo.py | Execute end-to-end synthetic workflows | Scenario requests | Console decisions and audit summary |
| Performance evaluation script | Measure correctness, speed, memory, clarity | Labeled workload | CSV results and PNG chart |
| Docker and configuration files | Package runtime and document settings | Source, dependencies, environment | Reproducible deployment baseline |

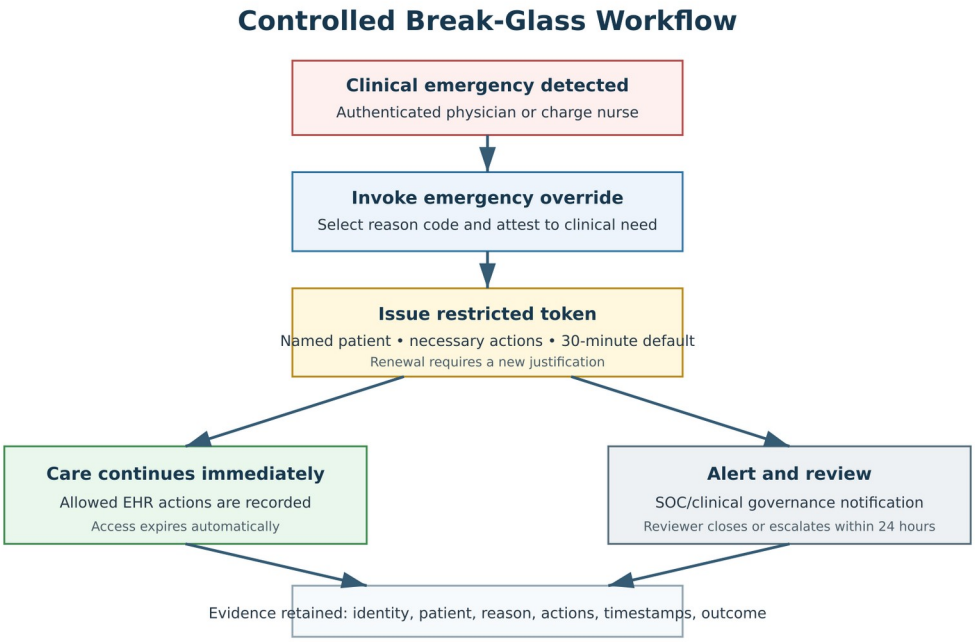
*Note.* RBAC = role-based access control; MFA = multifactor authentication; EHR = electronic health record; CSV = comma-separated values; PNG = Portable Network Graphics.

## 3.4 Design Diagrams

Figure 3 presents the controlled break-glass workflow, which is the most sensitive exception in the design. Table 6 documents the persisted audit-event entity, which is the data model shared by the engine, the repository, and the evaluation scripts.

**Figure 3**

*Controlled Break-Glass Workflow*




*Note.* Emergency access is patient-specific, time-limited, immediately logged, and subject to retrospective review. The 30-minute default shown is the logical design value; the prototype constant is 15 minutes.

**Table 6**

*Audit Event Data Model (audit_events Table)*

| Field | Type | Purpose |
|---|---|---|
| event_id | TEXT, primary key | Random 16-hex-character identifier returned to the caller |
| timestamp | TEXT | UTC ISO-8601 time of the decision |
| user_id | TEXT | Synthetic subject identifier (validated non-blank) |
| role | TEXT | One of the six implemented roles |
| action | TEXT | Requested EHR action |
| patient_id | TEXT | Synthetic patient identifier or None |
| decision | TEXT | PERMIT, DENY, STEP_UP_MFA, or BREAK_GLASS_PERMIT |
| reason | TEXT | Human-readable explanation of the outcome |
| checksum | TEXT | HMAC-SHA256 over the canonical payload |

*Note.* ID = identifier; hex = hexadecimal; UTC = Coordinated Universal Time; EHR = electronic health record; MFA = multifactor authentication; HMAC = keyed-hash message authentication code; SHA-256 = Secure Hash Algorithm with a 256-bit output. ISO 8601 specifies the date and time format. None indicates that no patient identifier is provided.

## 3.5 Functional and Non-Functional Requirements

Table 7 lists the requirements that guided implementation and the acceptance measure for each. They were refined from the Unit 3 catalog to match the implemented scope: the former lifecycle requirement (automated transfer and termination handling) was replaced by FR-02, which the prototype can verify, and its automation is retained as future work.

**Table 7**

*Functional and Non-Functional Requirements and Acceptance Measures*

| ID | Requirement | Acceptance measure and design influence |
|---|---|---|
| FR-01 | Enforce six-role least privilege | All prohibited role–action pairs are denied; central permission matrix and prohibited-action tests |
| FR-02 | Reject inactive, unauthenticated, or unidentified subjects | Each case returns DENY with a reason and an audit event; ordered validation guards |
| FR-03 | Apply step-up MFA to high-risk context | Export, account management, off-network, and unmanaged-device requests return STEP_UP_MFA without verified MFA |
| FR-04 | Evaluate patient and operational context | Missing patient identifier, off-duty status, or missing assignment is denied |
| FR-05 | Provide governed emergency access | Break-glass requires clinical role, patient, clinical action, and justification; returns expiry; always audited |
| FR-06 | Create complete, tamper-evident decision evidence | One HMAC-signed event per request persisted in SQLite; altered events fail verification |
| NFR-01 | Performance efficiency | Local mean latency and throughput baseline below the 250-ms policy-decision design target; conditions documented |
| NFR-02 | Reliability and recoverability | Regression suite passes on the released commit; backup, rollback, and restore activities in the maintenance plan |
| NFR-03 | Security and maintainability | Separation of duties, version control, configuration plan, pinned dependencies, non-root container |

*Note.* FR = functional requirement; NFR = non-functional requirement.

## 3.6 Ethical and Security Considerations

Privacy shaped the method: all staff, patient, department, and timestamp data are fictional, and no credentials, IP addresses, production screenshots, or health data were collected. Review participation was voluntary and the signed form explained the purpose and data used. Although the engine is rule-based, fairness still matters; the test set includes all roles, off-duty access, missing assignment, and emergency cross-cover, excludes protected traits from decisions, and gives reasons with a human-review route. Planned controls include encryption, least

privilege, managed secrets, session expiry, MFA, segregation of duties, protected audit retention, and fail-safe behavior. If the PDP is unavailable, routine access fails closed while approved downtime and emergency procedures preserve care. Production would require threat modeling, security testing, a privacy impact assessment, change control, and governance approval.

# CHAPTER 4: RESULTS AND EVALUATION

## 4.1 Implementation Overview

This chapter reports implementation, integration, testing, and evaluation. Units 4–7 progressed from the core engine to persistent integration, measurable outputs, system and acceptance tests, defect correction, deployment artifacts, and maintenance planning. The prototype uses only synthetic identifiers and no hospital connection. Its results demonstrate reproducible behavior within the implemented scope, not legal compliance, clinical safety, or production readiness.

### *4.1.1 Implementation Environment and System Baseline*

The repository separates source, tests, results, deployment, and documentation. The verified environment was Windows 11, Python 3.12.15, pytest 9.1.1, and Matplotlib 3.11.2. Commit d2ac7cd and tag v1.2.0-unit7 passed all 25 tests on October 17, 2026. The final release aligned the pytest dependency pin and reran the complete suite on October 24, 2026; all 25 tests again passed on commit 0bb3017 (Section 4.4). Thus, the reported evidence and release source identify the same verified states.

### *4.1.2 Core Algorithm Implementation Results*

The engine applies account, authentication, and subject checks before emergency or routine authorization. Normal requests then pass through role, patient, duty, assignment, and risk

checks. This order supports least privilege, prevents the emergency flag from bypassing identity, and returns an inspectable reason. Six role baselines are narrowed by patient assignment, duty, network, device, and authentication context, addressing the limits of static roles observed in hospital access logs (Røstad & Edsberg, 2006). STEP_UP_MFA permits reauthentication and resubmission (Temoshok et al., 2025); BREAK_GLASS_PERMIT includes a 15-minute expiry and signed evidence. A test confirms that changing a signed field makes verification fail. The release uses a synthetic HMAC key that production must replace with a managed secret.

The excerpt below, from src/access_control_engine.py at the released commit, shows the guard order that the tests verify; every branch exits through _finish, which creates the reasoned decision and the signed audit event.

```
def evaluate(self, request: AccessRequest) -> DecisionResult:
    if not request.account_active:
        return self._finish(request, AccessDecision.DENY,
                            "Account is inactive.")
    if not request.user_id or not request.user_id.strip():
        return self._finish(request, AccessDecision.DENY,
                            "A valid user identifier is required.")
    if request.emergency:
        return self._evaluate_break_glass(request)
    ...  # role, patient identifier, duty period, assignment guards
    if elevated_risk and not request.mfa_verified:
        return self._finish(request, AccessDecision.STEP_UP_MFA,
    "Additional MFA verification is required for this context.")
    return self._finish(request, AccessDecision.PERMIT,
                        "Role and contextual access checks passed.")
```

### *4.1.3 System Integration and Module Communication*

EHRAccessService is the application boundary. It receives an AccessRequest, calls the engine once, finds the signed event created for that decision, persists it through AuditRepository, and returns DecisionResult. This keeps policy logic independent from connection handling and allows future replacement of SQLite without changing the caller. The integrated path is client → service → engine → repository → SQLite → response (Appendix E). Each request, including denials and MFA challenges, creates one committed row through a parameterized INSERT.

SQLite is suitable for this transactional prototype; only synthetic security metadata is stored, never clinical content or real identifiers.

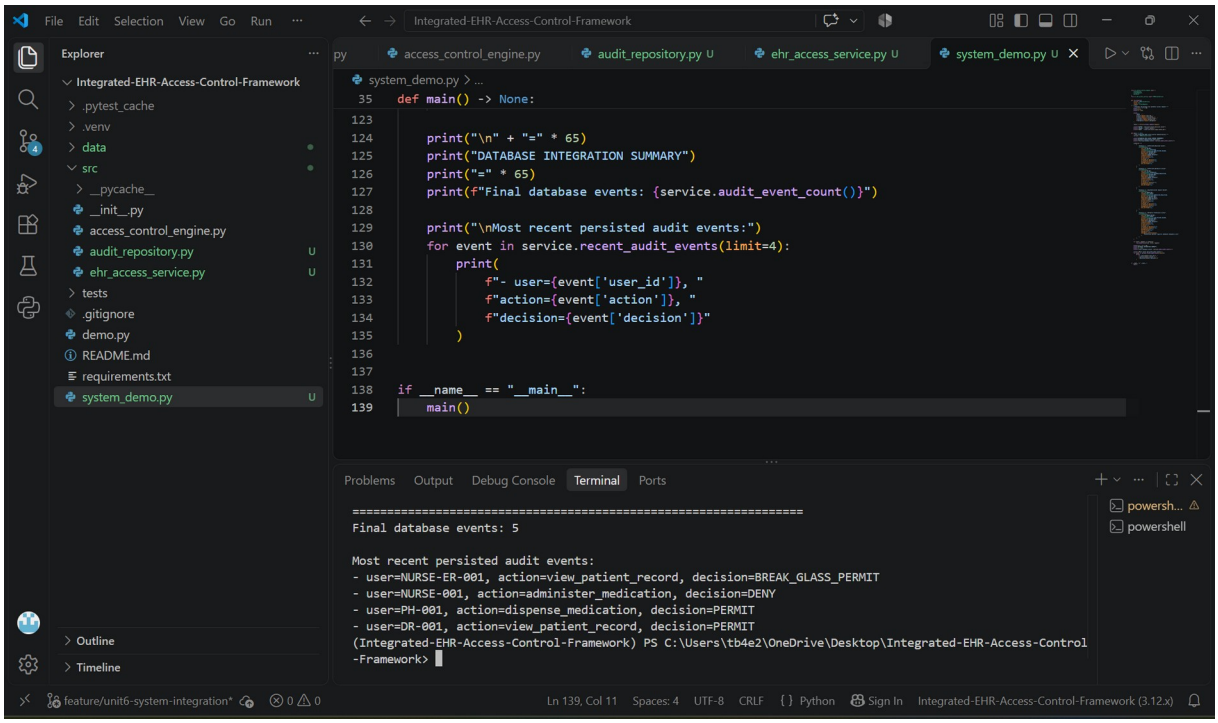
## 4.2 Output and Screenshots

### *4.2.1 End-to-End Demonstration*

The demonstration processes four workflows: assigned physician viewing, pharmacist dispensing, unauthenticated nurse access, and justified nurse break-glass access. Figure 4 shows PERMIT for the first two, DENY for the third, and BREAK_GLASS_PERMIT for the fourth. The database count was five because one earlier event remained in the demonstration file; the four newest rows match the four scenarios. Retrieval after persistence therefore validates communication among the driver, service, engine, repository, and database.

**Figure 4**

*End-to-End Demonstration and Persisted Audit Summary*




*Note.* The console displays the persisted event count and the four newest decisions (BREAK_GLASS_PERMIT, DENY, PERMIT, PERMIT) retrieved from SQLite after the scenarios ran.

### *4.2.2 Testing Levels and Procedures*

The suite contains 15 unit tests, three integration tests, and seven Unit 7 system, acceptance, and stability tests. Unit tests isolate policy branches; integration tests submit permit, deny, and break-glass requests through the real service and temporary repository; and higher-level tests exercise complete workflows plus a sequential 50-request stability case. Temporary paths isolate databases. The focused Unit 7 file ran first, followed by the complete regression suite; the verbose log is retained in results/unit7_full_test_log.txt.

A key integration test submits an unassigned nurse emergency request with justification and verifies BREAK_GLASS_PERMIT, one persisted event, and matching user and decision fields. It protects against an engine that returns the correct emergency result while persistence fails or records a different outcome. Table 8 summarizes the seven Unit 7 cases.

**Table 8**

*Unit 7 System, Acceptance, and Stability Test Cases*

| Case and level | Expected behavior | Observed result |
|---|---|---|
| Blank subject (system) | DENY; reason mentions the user identifier; one audit event | Initially failed; passed after the validation guard |
| Assigned physician (acceptance) | PERMIT for the assigned patient | Passed |
| High-risk export (acceptance) | STEP_UP_MFA without verified step-up authentication | Passed |
| Break-glass (acceptance) | Time-limited permit plus a persistent audit event | Passed |
| IT administrator (acceptance) | DENY for default clinical-record access | Passed |
| Mixed workflow (system) | One complete signed audit event per request | Passed |
| Fifty sequential requests (stability) | All requests processed; no event loss | Passed |

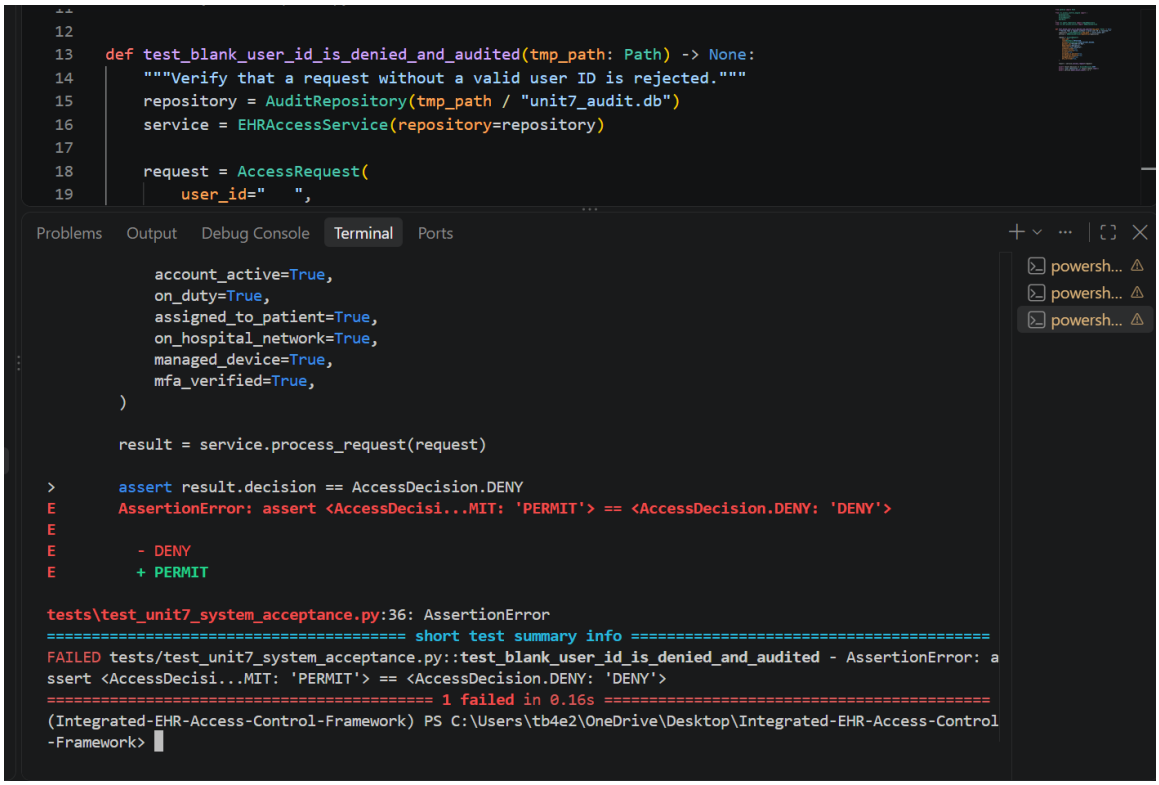
*Note.* IT = information technology; MFA = multifactor authentication. PERMIT and DENY indicate access approval and denial, respectively. STEP_UP_MFA indicates that additional authentication is required. Break-glass refers to controlled emergency access.

### *4.2.3 Defect Discovery and Resolution*

The new system tests initially exposed one security-relevant failure: a user_id containing only spaces reached PERMIT because the engine checked account and authentication state but not a nonblank identifier (Figure 5). Such an event cannot provide reliable attribution and therefore must be denied.

**Figure 5**

*Initial Failed Blank-Identifier System Test*



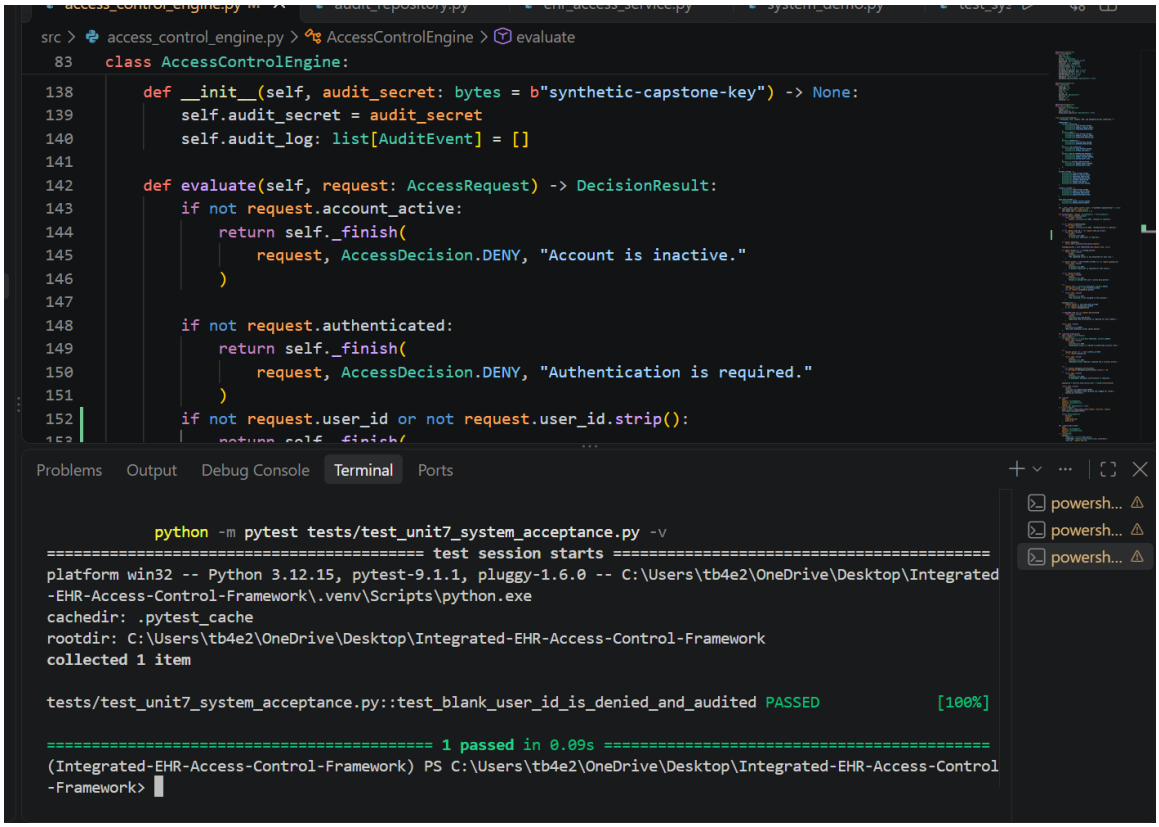

*Note.* The observed PERMIT contradicted the required DENY outcome and led directly to the validation correction.

The fix checks both a missing value and user_id.strip() before role or emergency logic. It uses the existing _finish method so the denial remains audited. The focused test passed (Figure 6), followed by all 25 regression tests (Figure 7). This fail–fix–retest sequence connects a requirement, observed defect, code change, and verified outcome.

```
if not request.user_id or not request.user_id.strip():
    return self._finish(
        request,
        AccessDecision.DENY,
        "A valid user identifier is required.",
    )
```

**Figure 6**

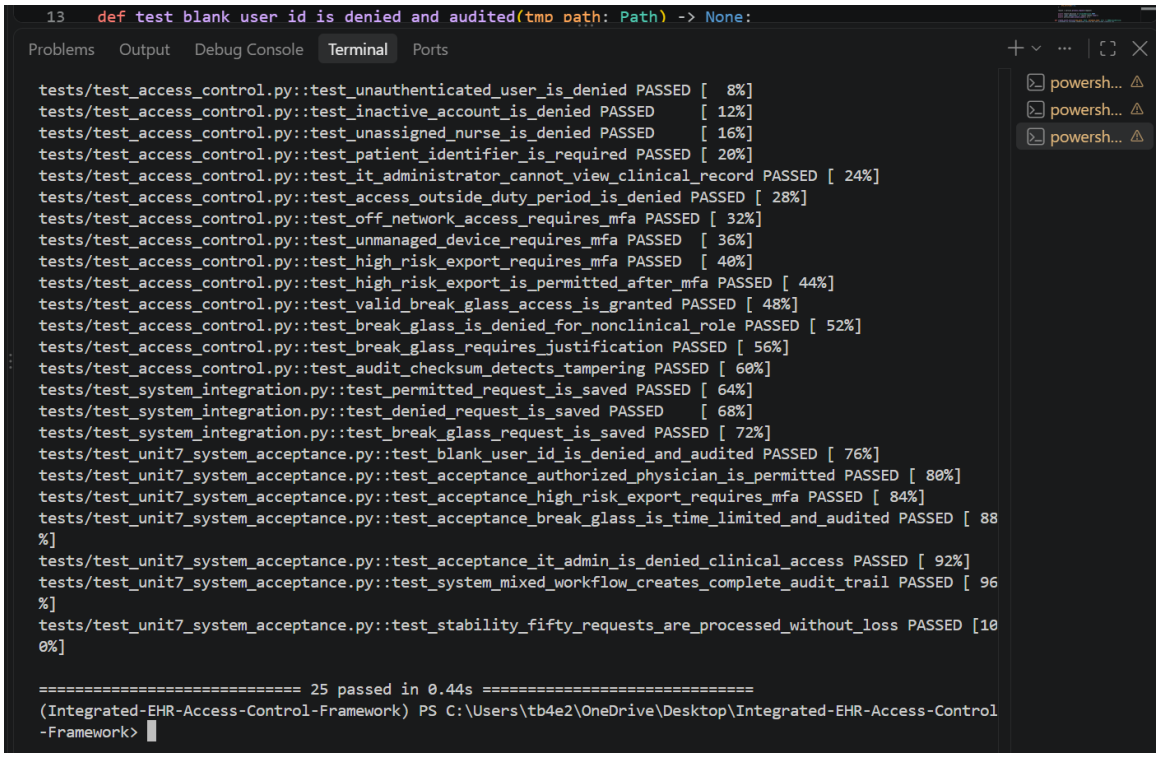
*Corrected Validation and Focused Passing Test*




*Note.* The engine denies a blank subject while preserving the standard audited return path; the targeted test reports one pass.

**Figure 7**

*Complete 25-Test Regression Result*




*Note.* Fifteen unit tests, three integration tests, and seven Unit 7 system, acceptance, and stability tests completed successfully in 0.44 seconds.

### *4.2.4 Scope and Limits of the Tests*

The tests cover inactive, unauthenticated, and blank subjects; unauthorized roles; patient assignment; duty; MFA escalation; emergency justification and persistence; audit completeness; and signed-event tampering. They do not cover a real identity provider, MFA bypass, stolen-key recovery, secret rotation, concurrency, network or TLS attacks, denial of service, vulnerability scanning, or penetration testing. The 50-request case is sequential, and acceptance tests are automated rather than clinician sessions. Thus, 25 passes establish conformance only for the documented synthetic cases.

## 4.3 Evaluation Metrics

### *4.3.1 Benchmark Conditions*

The benchmark is a documented local sequential baseline. Both runs used one Windows 11 workstation (build 10.0.26200), a 12th-generation Intel processor with 12 logical CPUs, 7.7 GB usable memory, and Python 3.12.15 in one process; the final script saves these details in results/evaluation_environment.json. Accuracy used four prelabeled cases in Unit 6 and 36 in the final run, including a count of false permits. Performance measurement used ten warm-up requests and 300 sequential physician requests through the engine, service, SQLite INSERT, and commit. The final run timed each request with time.perf_counter and reported mean, median, nearest-rank p95 and p99, elapsed-time throughput, and tracemalloc peak incremental allocation. A temporary database was deleted afterward. Operating-system load, caching, and power state were uncontrolled, so the results are a regression baseline and do not predict concurrent hospital performance.

### *4.3.2 Quantitative and Qualitative Results*

The script saved CSV, environment, and chart outputs. Unit 6 achieved 100% accuracy on four cases, mean latency of 3.654–3.704 ms, throughput of 269.96–273.68 requests/s, 0.117 MB peak memory, and 4.60/5 clarity. In the final run, all 36 cases matched expectations with zero false permits. Mean latency was 3.196 ms, median 3.131 ms, p95 3.976 ms, p99 4.205 ms, and the range 2.564–4.500 ms; throughput was 312.18 requests/s and peak incremental memory was 0.127 MB. Because the decision path did not change, the difference reflects timing and workstation variability, not an algorithmic improvement. Every observed request remained far below the 250-ms target. Table 9 and Figure 8 present the results.

**Table 9**

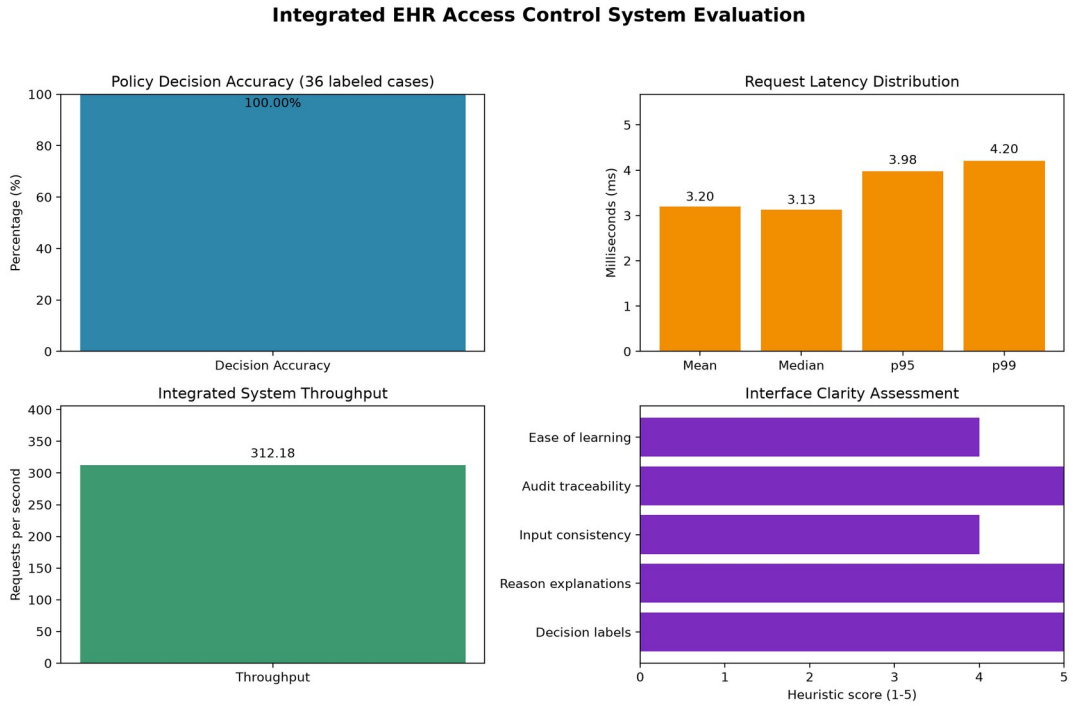
*Integrated Evaluation Results*

| Measure | Observed result | Interpretation | Status |
|---|---|---|---|
| Decision accuracy | 100.00% (4/4) in Unit 6; 100.00% (36/36) with 0 false permits in the final run | Labeled decisions matched; no false permits | Target met (≥95%, zero false permits) |
| Mean latency | 3.654–3.704 ms in Unit 6; 3.196 ms in the final run | Mean local integrated request time, including the SQLite write | Below the 250-ms design target |
| Median, p95, and p99 latency | Not recorded in Unit 6; 3.131, 3.976, and 4.205 ms in the final run | Distribution of the 300 timed requests; p99 about 60 times below target | NFR-01 met locally |
| Throughput | 269.96–273.68 requests/s in Unit 6; 312.18 requests/s in the final run | Sequential local requests per second | Baseline established |
| Peak memory | 0.117 MB in Unit 6; 0.127 MB in the final run | Peak incremental allocation during the timed block | Low local footprint |
| Interface clarity | 4.60/5 (unchanged) | Developer heuristic across five stated criteria | Above the 4.0/5 target |
| Expert validation | 4.85/5 (n = 2); external reviewer alone 4.80/5 | Ten-statement review by two healthcare IT experts (Section 4.3.3) | Above the 4.0/5 target |
| Automated tests | 25/25 passed on d2ac7cd and on 0bb3017 | Unit, integration, system, acceptance, and stability | Complete pass |

*Note.* ms = milliseconds; MB = megabytes; requests/s = requests per second; p95 and p99 = 95th and 99th latency percentiles, respectively; NFR = non-functional requirement; IT = information technology; *n* = number of expert reviewers. Performance measurements reflect local sequential testing. Final-run accuracy was assessed using 36 labeled cases, whereas latency statistics summarize 300 timed requests. Alphanumeric identifiers denote the tested Git commits.

**Figure 8**

*Integrated System Evaluation Chart*




*Note.* The chart (results/system_evaluation_chart.png) reports decision accuracy over the 36 labeled cases, the latency distribution (mean, median, p95, p99), sequential throughput, and the five interface-clarity criteria from the final evaluation run on commit 0bb3017.

The clarity heuristic rated decision labels, explanations, and audit traceability at 5/5 and input consistency and ease of learning at 4/5, for a mean of 4.60. The disclosed calculation is inspectable but is not a validated usability instrument; the expert review adds independent practitioner feedback.

### *4.3.3 Expert Reviewer Validation*

Two healthcare IT experts reviewed the prototype on October 24, 2026, using Appendix D. Reviewer 1, an independent healthcare information security consultant, has 12 years of experience. Reviewer 2, a health informatics and IT operations manager with 14 years of

experience, is the author's brother; this potential conflict is disclosed, and his ratings are reported separately. He did not design, implement, or test the system. Both reviewers used the same public portfolio, exercised the four demonstration scenarios plus at least two variations, and rated ten statements covering security behavior, clarity, and applicability.

**Table 10**

*Expert Reviewer Ratings on the Ten Validation Statements*

| Statement | Reviewer 1 | Reviewer 2 | Mean |
|---|---|---|---|
| C1 Role permissions follow least privilege and match hospital work | 5 | 5 | 5.00 |
| C2 Contextual checks are appropriate for a hospital EHR | 5 | 5 | 5.00 |
| C3 Step-up MFA is required in the right situations | 5 | 5 | 5.00 |
| C4 Break-glass path is controlled appropriately | 5 | 5 | 5.00 |
| C5 Audit trail supports accountability; tamper detection adds value | 5 | 5 | 5.00 |
| C6 Decision labels are clear and unambiguous | 4 | 5 | 4.50 |
| C7 Decision reasons are understandable to non-technical staff | 5 | 5 | 5.00 |
| C8 Inputs are consistent and the system is easy to learn | 5 | 4 | 4.50 |
| C9 Audit traceability: who requested what, when, and the decision | 5 | 5 | 5.00 |
| C10 Applicable in a Saudi government hospital after development | 4 | 5 | 4.50 |
| Security behavior (C1–C5) | 5.00 | 5.00 | 5.00 |
| Usability and clarity (C6–C9) | 4.75 | 4.75 | 4.75 |
| Applicability (C10) | 4.00 | 5.00 | 4.50 |
| Overall mean (C1–C10) | 4.80 | 4.90 | 4.85 |

*Note.* Scale: 1 = strongly disagree, 5 = strongly agree. Success criterion: mean ≥ 4.0 with no unresolved critical concern (Table 2). Both reviewers answered the pilot question with "Yes, with the changes above."

Security behavior received 5.00/5 from both reviewers, usability and clarity averaged 4.75, and applicability 4.50, for an overall mean of 4.85. The external reviewer alone averaged 4.80. Both values exceed the predefined 4.0 threshold, so O5 is met without relying on the

related reviewer's score. Decision-label clarity (C6) and ease of learning (C8) each received one rating of 4, consistent with the developer heuristic. Both reviewers recommended a pilot with changes and reported no critical rule that would block legitimate care or permit inappropriate access.

Both reviewers praised context-aware least privilege, controlled 15-minute break-glass access, and tamper-evident auditing. Reviewer 1 noted legacy-EHR integration and possible MFA or network latency, recommending concurrency testing, alignment with national identity services such as Nafath, and a break-glass review dashboard. Reviewer 2 noted clinician orientation and directory dependence, recommending degraded-network caching, role-specific micro-training, and SIEM export. Sections 5.3 and 6.4 incorporate these recommendations, and Appendix D reproduces the signed forms.

#### *4.3.4 Requirements Traceability and Objective Status*

Table 11 links each requirement to the automated tests and other evidence that verify it, satisfying the 100% traceability criterion. Table 12 then reports the status of each objective against the success criteria defined in Chapter 1; with the final evaluation and the expert validation complete, every objective is met.

**Table 11**

*Requirements Traceability Matrix*

| ID | Verifying tests and evidence | Status |
|---|---|---|
| FR-01 | test_assigned_physician_is_permitted; test_it_administrator_cannot_view_clinical_record; test_acceptance_it_admin_is_denied_clinical_access; role matrix (Table 4) | Met |
| FR-02 | test_unauthenticated_user_is_denied; test_inactive_account_is_denied; test_blank_user_id_is_denied_and_audited; test_denied_request_is_saved | Met (after Unit 7 correction) |
| FR-03 | test_off_network_access_requires_mfa; test_unmanaged_device_requires_mfa; test_high_risk_export_requires_mfa; test_high_risk_export_is_permitted_after_mfa; test_acceptance_high_risk_export_requires_mfa | Met |
| FR-04 | test_unassigned_nurse_is_denied; test_patient_identifier_is_required; test_access_outside_duty_period_is_denied | Met |
| FR-05 | test_valid_break_glass_access_is_granted; test_break_glass_is_denied_for_nonclinical_role; test_break_glass_requires_justification; test_break_glass_request_is_saved; test_acceptance_break_glass_is_time_limited_and_audited | Met |
| FR-06 | test_audit_checksum_detects_tampering; test_permitted_request_is_saved; test_system_mixed_workflow_creates_complete_audit_trail; test_stability_fifty_requests_are_processed_without_loss | Met |
| NFR-01 | Benchmark of 300 sequential requests with mean, median, p95, and p99 (Table 9); conditions and hardware in Section 4.3.1 | Met locally (p99 4.205 ms against the 250-ms target) |
| NFR-02 | 25/25 regression on d2ac7cd and 0bb3017; maintenance schedule (Table 13) | Met for the prototype; restore drills planned |
| NFR-03 | Dockerfile with non-root user, dependencies pinned to the verified versions, .env.example, branch and tag workflow | Met for the prototype |

*Note.* ID = identifier; FR = functional requirement; NFR = non-functional requirement; MFA = multifactor authentication; ms = milliseconds. p95 and p99 denote the 95th and 99th latency percentiles, respectively. Entries beginning with test_ identify automated test functions. Status reflects verification within the evaluated prototype scope.

**Table 12**

*Objective Status Against Success Criteria*

| Objective | Evidence | Status |
|---|---|---|
| O1: Least-privilege matrix | Six-role matrix implemented and tested; prohibited pairs denied | Met |
| O2: Testable requirements | Nine requirements with acceptance measures; 100% FR traceability | Met |
| O3: Executable prototype with signed evidence | Integrated service, SQLite persistence, HMAC verification, 25 tests | Met |
| O4: Multi-level testing and benchmark | 25/25 tests; 100% accuracy on 36 labeled cases with zero false permits; documented benchmark with percentiles and hardware | Met |
| O5: Structured usability feedback | Heuristic 4.60/5; two-reviewer validation mean 4.85/5, external reviewer alone 4.80/5 (Table 10; Appendix D) | Met |

*Note.* O1–O5 denote the five project objectives; FR = functional requirement; HMAC = keyed-hash message authentication code. “Met” indicates that the stated success criteria were satisfied within the evaluated prototype scope. Heuristic and expert-review scores are reported out of 5.

## 4.4 Deployment and Version-Control Evidence

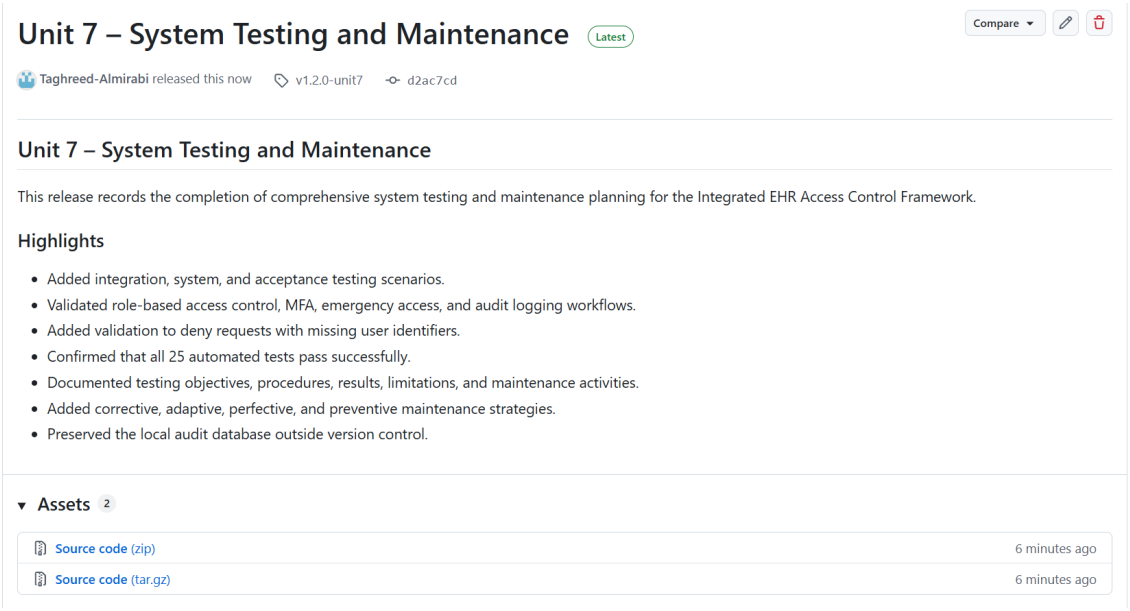
The deployment package contains requirements.txt, Dockerfile, .dockerignore, .env.example, and deployment/DEPLOYMENT.md. The image installs pinned dependencies, copies the required files, runs as non-root ehruser, declares persistent storage, and starts system_demo.py. The example environment file documents the configuration names without production secrets. The current constructor still supplies synthetic defaults for the database and signing key; wiring validated environment values into the entry point remains an adaptive-maintenance task.

Git preserves traceability. Unit 7 commits ed9789c and f43e315 moved from the feature branch through development (46e1e7d) to main (d2ac7cd), where all 25 tests passed. Release

v1.2.0-unit7 targets d2ac7cd (Figure 9); v1.1.0-unit6 targets a4fbbfe, and v1.0.0 remains the Unit 5 baseline at cc2766a. Release notes record tests, the identifier fix, synthetic-data scope, and limitations. This tag-to-commit link provides an identifiable source state for the evidence, consistent with SSDF release traceability (Souppaya et al., 2022).

**Figure 9**

*Verified Unit 7 GitHub Release*



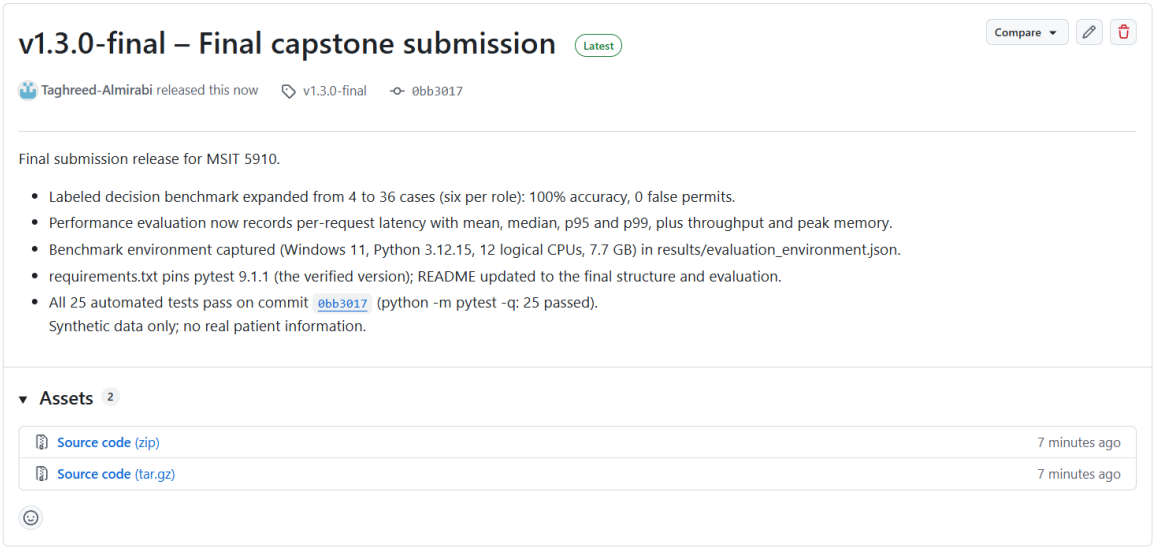

*Note.* The latest release links tag v1.2.0-unit7 to merge commit d2ac7cd and records the 25-test milestone.

The final evaluation and dependency update were committed to main as 0bb3017 on October 24, 2026, after all 25 tests passed. Release v1.3.0-final points to that commit and records the 36-case benchmark, percentiles, environment, and dependency alignment (Figure 10). Because the engine, service, and repository did not change after d2ac7cd, the Unit 7 tests remain applicable, Appendix A reproduces the commands, and the two releases provide verified rollback points. The public repository, the release pages, and the project portfolio, which hosts the report,

the presentation slides, and the recorded demonstration video, are listed with their links in Appendix C.

**Figure 10**

*Verified Final Release v1.3.0-final*




*Note.* The final release links tag v1.3.0-final to commit 0bb3017 and records the 36-case benchmark, the latency percentiles, the environment capture, the dependency alignment, and the 25-test result.

# CHAPTER 5: DISCUSSION

## 5.1 Interpretation of Results

The results demonstrate an integrated, traceable request path: identity and context checks, emergency handling, audit integrity, persistence, testing, configuration, and release evidence work together. Each action produces an explained decision and signed, stored event. Within the prototype boundary, this answers RQ1 and RQ2: stable roles plus limited context remained understandable, while normal, high-risk, and emergency outcomes were enforced and audited.

All 36 labeled cases matched their predefined rules, and 25 automated tests added persistence and integrity coverage. This supports internal validity but measures synthetic rule conformance, not unknown hospital workflows. Mean latency near 3 ms, p99 of 4.2 ms, and sequential throughput above 300 requests/s were well within the 250-ms design target. Because the benchmark excludes network, identity-provider, encryption, contention, and concurrent load, it is a regression baseline rather than a hospital capacity guarantee.

### *5.1.1 Quantitative Evaluation*

Accuracy, latency, throughput, and incremental memory address different quality properties, consistent with ISO/IEC 25010:2023 (ISO, 2023). Labeled cases validate policy outcomes, unit tests isolate branches, and integration tests verify database effects. Reporting both runs avoids false precision: the final mean was about 14% faster than the earlier capture, although both were far below the target. The final environment record and percentiles improve reproducibility, but repeated trials and concurrent tests are still needed.

### *5.1.2 Qualitative Evaluation and User Experience*

Because a graphical clinical interface was outside scope, the heuristic evaluated command-line interaction. Labels, reasons, and audit identifiers were clear, while users still needed to understand Python fields and flags. A real EHR would derive assignment and emergency context rather than asking clinicians to set them. The experts similarly rated explanations at 5.00 and labels at 4.50, locating remaining friction in learning inputs and interpreting step-up prompts during busy work.

## 5.2 Comparison with Literature Findings

Cobrado et al. (2024) found recurring gaps in EHR implementation, emergency access, MFA, consent, and accountability. The prototype does not address consent, but it combines the other elements: RBAC establishes permissions, context limits overreach, MFA raises assurance, break-glass preserves availability, and signed audits support accountability. This retains administrable roles (Ferraiolo et al., 2001) while avoiding open-ended attribute policies that are hard to audit (Hu et al., 2014). It also responds to Saudi confidentiality concerns (Almaghrabi & Bugis, 2022) and to evidence that real clinical access needs exceed static assignments (Røstad & Edsberg, 2006).

Contextual healthcare authentication research supports using user, environment, device, and use-case data (Suleski & Ahmed, 2023), and NIST supports step-up authentication when a higher assurance level is required (Temoshok et al., 2025). Break-glass follows the principle that exceptional access should be explicit and logged (Brucker & Petritsch, 2009). The engine also evaluates each request instead of trusting network location, consistent with zero trust (Rose et al., 2020; Vukotich, 2023), although it lacks continuous risk and device-posture signals.

The design maps to NIST access, authentication, audit, configuration, and integrity controls (Joint Task Force, 2020), log-management guidance (Kent & Souppaya, 2006), and Saudi ECC-2:2024 (NCA, 2024). Code alone does not establish compliance, which also requires policy, ownership, monitoring, and risk treatment. PDPL principles supported synthetic and minimized data use (SDAIA, 2023); realistic data would require authorization, purpose, retention, access limits, and privacy review.

Reviewer priorities align with the literature: national identity integration supports assurance guidance (Temoshok et al., 2025); latency concerns reflect the availability problem addressed by break-glass research (Brucker & Petritsch, 2009); SIEM export and an emergency-review dashboard operationalize log-management guidance (Kent & Souppaya, 2006); and training addresses implementation gaps identified across EHR studies (Cobrado et al., 2024). Their independent emphasis on these issues provides modest face-validity evidence.

## 5.3 Implications and Limitations

The roles and scenarios derive from literature and project assumptions rather than an approved hospital policy catalog. Real authorization also includes specialties, delegation, trainees, researchers, external providers, consent, and temporary assignments. The final submission closed two planned gaps by expanding the benchmark to 36 cases and obtaining two expert reviews; automated joiner–mover–leaver processing remains represented only by flags. The reviewer sample establishes face validity, not statistical generalizability, and should expand to clinicians, HIM, privacy, and compliance personnel. One reviewer is the author's brother, creating potential bias despite his independent use of the public demonstration; the relationship and his separate score are disclosed, and the external reviewer's 4.80 mean also exceeds the

threshold. Both reviews used demonstrations rather than live workflows. The prototype also omits legacy integration, degraded-network behavior, clinician training, real identity services, credential attacks, stolen keys, concurrency, vulnerability scanning, and penetration testing. One workstation, SQLite, a developer clarity heuristic, and no repeated-trial confidence intervals further limit generalization. The findings support reproducibility within scope, not hospital capacity, security authorization, clinical safety, or user acceptance.

### *5.3.1 Deployment Strategy*

Deployment options include direct on-premise Python, managed cloud, and containers. Direct installation increases environment drift; cloud use requires approved residency, contracts, identity integration, and governance; Docker provides reproducibility without dictating location. The selected approach is a container on a hospital-controlled Linux server or approved Saudi private cloud. The image uses a slim base, pinned packages, non-root execution, and persistent storage, consistent with NIST container guidance (Souppaya et al., 2017). Production promotion should scan and sign an image pinned by digest, then move through development, test, staging, and a monitored read-only pilot with rollback.

### *5.3.2 Configuration Management and Reliability*

Explicit configuration improves reliability. .env.example documents deployment stage, database path, log level, signing key, and synthetic-data mode without secrets; requirements.txt pins packages; the Dockerfile defines the runtime; and DEPLOYMENT.md records commands and cautions. The entry point should validate these variables and fail fast when production values are absent. A secrets manager must inject and rotate the audit key. SQLite should be replaced by an encrypted, highly available managed database with monitored backup, recovery, retention,

and defined audit-write failure behavior. CI can run tests, scans, and container builds, while protected branches and tagged releases preserve tested rollback points.

#### *5.3.3 Security, Privacy, and Ethical Considerations*

Least privilege, contextual checks, step-up MFA, governed break-glass access, and signed logs reduce but do not eliminate risk. Production emergency access should notify an accountable reviewer, require timely retrospective review, detect repeated use, and define consequences for misuse. Synthetic data avoided patient privacy exposure, and screenshots and repository contents were checked for credentials and clinical data. Reviewers signed informed declarations. Any future realistic data must be lawfully sourced, minimized or de-identified, access-controlled, and retained only for an approved purpose.

## 5.4 Maintenance and Operational Sustainability

Maintenance is part of the security model. Corrective work addresses confirmed defects; adaptive work covers dependency, identity, EHR API, platform, regulatory, and configuration changes; perfective work improves explanations, accessibility, administration, performance, and workflow; and preventive work includes refactoring, scanning, key rotation, access recertification, and restore tests (ISO, 2022a). Aligning pytest to 9.1.1 in v1.3.0-final is one adaptive example. Each change begins with an issue containing severity, requirement, reproduction evidence, acceptance criteria, and owner, then proceeds on a reviewable feature or hotfix branch with tests and documentation. A failed security check blocks merge and is remediated and rerun on the same branch. Protected tags preserve milestones and rollback evidence, consistent with NIST configuration and secure-development guidance (Joint Task Force, 2020; Souppaya et al., 2022). Table 13 gives the schedule.

**Table 13**

*Proposed Maintenance and Monitoring Schedule*

| Frequency | Activity | Primary evidence |
|---|---|---|
| Every commit or pull request | Run the full test suite; review policy, schema, dependency, and configuration changes. | CI log, review record, linked issue |
| Weekly | Review authentication failures, denials, emergency events, checksum alerts, and open defects. | Operations and security review |
| Monthly | Patch dependencies; scan source and image; compare errors, latency, memory, and storage growth. | Patch and metric report |
| Quarterly | Recertify roles; test backup restore, secret rotation, and rollback; review break-glass use. | Signed governance record |
| Annually or after major change | Refresh threat model, privacy impact assessment, architecture, recovery objectives, and user training. | Multidisciplinary approval |

## 5.5 Post-Deployment Risks and Mitigation

Table 14 is the project's post-deployment risk register; each risk is paired with its likely effect and with mitigation or contingency measures tied to the maintenance schedule and the configuration-management rules above.

**Table 14**

*Post-Deployment Risks and Controls*

| Risk | Likely effect | Mitigation and contingency |
|---|---|---|
| Vulnerable dependency or image | Unauthorized access or service compromise | Scanning, risk-based patch timelines, signed image, incident response, rollback to the last verified release |
| Policy or privilege drift | Excessive access or false denial | Versioned policy, separation of duties, quarterly recertification, regression cases |
| Audit inconsistency, tampering, or growth | Missing evidence, failed writes, exhausted storage | Transactional store, checksum verification, centralized logging, retention, alerts, backup and restore drills |
| Emergency-access misuse | Privacy breach or insider abuse | Justification, expiry, alert, retrospective review, trend monitoring, sanctions |
| Configuration drift | Different behavior across environments | Approved baseline, secrets manager, staged promotion, health checks, tested rollback |
| Usability-driven workarounds | Shared credentials, bypass attempts, care delay | Clinician testing, clear reasons, training, support metrics, controlled rule refinement |
| Directory or network dependency (reviewer finding) | Context checks unavailable or slow; decisions delayed during network degradation | Cached role and assignment context with an audited cache window; fail-closed default with the approved downtime and emergency procedure; latency percentiles tracked per release |

## 5.6 Overall Interpretation

The prototype is internally coherent: modules exchange expected objects, decisions persist, tests pass, the discovered defect was corrected without regression, and resource use remained small. Executable behavior, release traceability, and maintenance evidence make it stronger than a design-only proposal, but simulated identity, workflow, infrastructure, and human factors keep it below pilot evidence. RQ3 therefore supports advancement to a controlled nonproduction integration environment under the conditions identified by the reviewers, not unrestricted deployment.

# CHAPTER 6: CONCLUSION AND FUTURE WORK

## 6.1 Summary of Main Findings

The project produced an integrated EHR access-control prototype with six-role RBAC, clinical context, step-up MFA, break-glass access, signed audits, persistence, layered tests, deployment artifacts, and maintenance planning. All 25 tests passed on the verified releases; 36 cases achieved 100% accuracy and zero false permits; mean and p99 latency were 3.196 and 4.205 ms; throughput was 312.18 requests/s; peak incremental memory was 0.127 MB; clarity scored 4.60/5; and two experts rated the framework 4.85/5 and recommended a controlled pilot. A blank-identifier defect was discovered, corrected, and regression-tested.

## 6.2 Implications and Project Contribution

The main contribution is a practical bridge between separate security recommendations and one executable healthcare example. The prototype shows how role, context, authentication, emergency access, persistence, and review can cooperate without implementing an entire EHR. It also offers a reusable evaluation pattern: predefined policy cases, integration tests for persistence, documented local metrics, requirement traceability, and realistic deployment and maintenance plans. In the Saudi context, synthetic data, least privilege, purpose-related context, and audited exceptions demonstrate privacy by design while making no certification claim.

## 6.3 Lessons Learned and Challenges

Integration revealed assumptions hidden by unit code. The final system required a clear transaction boundary, durable storage, retrieval, repeatable configuration, and evidence that emergency decisions reached the database; EHRAccessService supplied that boundary without duplicating policy logic. Benchmark variation showed why hardware, method, percentiles, and repeated trials must accompany performance claims. The whitespace-only identifier failure showed that system tests must challenge attribution assumptions, not only rule branches. Other challenges included consistent imports, isolated temporary databases, locked result files, configuration separation, and disciplined branching and releases. Finally, expert feedback showed that practitioners accepted the rules but questioned dependencies, latency, training, and degraded operation—conditions that tests must model in the next stage.

## 6.4 Future Work

The next stage should prioritize the reviewers' recommendations: concurrent and load testing at emergency-department volumes; nonproduction integration with a national identity service such as Nafath at an appropriate assurance level; a governed break-glass review dashboard; audited caching for directory or network degradation; role-specific guidance for step-up and denial messages; and SIEM-standard audit export. The service should then expose an authenticated API, load validated environment settings, replace hard-coded policy data with a versioned policy store, model consent, and use an encrypted managed database with centralized alerting. Testing should add concurrency, failure injection, malformed requests, key rotation, property-based invariants, dependency and container scans, static analysis, and independent penetration testing. Clinicians, HIM staff, privacy professionals, and security reviewers should

complete approved workflow studies with a validated usability instrument before any real patient data are used.

Emergency-session closure also needs refinement. The prototype expires break-glass access after 15 minutes but lacks an earlier inactivity timeout and an explicit close action. NIST distinguishes overall and inactivity timeouts and recommends no more than 15 minutes of inactivity at the highest assurance level (Temoshok et al., 2025). A future version should support inactivity expiry, explicit closure, automatic revocation and reauthentication, and a signed termination event, with values governed by hospital policy rather than fixed code.

## 6.5 Conclusion

The capstone transformed an access-control proposal into a working, testable, measurable, and maintainable prototype for controlled evaluation. Its strongest result is the traceable chain from request to explained decision, persisted audit evidence, test result, and verified release. Its strongest limitation is the synthetic, single-machine setting and two-person expert review rather than clinical use. The artifact is therefore a credible foundation for stakeholder review and nonproduction integration, but it still requires substantial security, privacy, usability, and operational validation before processing real patient data.

# REFERENCES

Almaghrabi, N. S., & Bugis, B. A. (2022). Patient confidentiality of electronic health records: A recent review of the Saudi literature. *Dr. Sulaiman Al Habib Medical Journal*, *4*(3), 126–135. https://doi.org/10.1007/s44229-022-00016-9

Brucker, A. D., & Petritsch, H. (2009). Extending access control models with break-glass. In *Proceedings of the 14th ACM Symposium on Access Control Models and Technologies* (pp. 197–206). Association for Computing Machinery. https://doi.org/10.1145/1542207.1542239

Cobrado, U. N., Sharief, S., Regahal, N. G., Zepka, E., Mamauag, M., & Velasco, L. C. (2024). Access control solutions in electronic health record systems: A systematic review. *Informatics in Medicine Unlocked*, *49*, Article 101552. https://doi.org/10.1016/j.imu.2024.101552

Fernández-Alemán, J. L., Señor, I. C., Lozoya, P. Á. O., & Toval, A. (2013). Security and privacy in electronic health records: A systematic literature review. *Journal of Biomedical Informatics*, *46*(3), 541–562. https://doi.org/10.1016/j.jbi.2012.12.003

Ferraiolo, D. F., Sandhu, R., Gavrila, S., Kuhn, D. R., & Chandramouli, R. (2001). Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security*, *4*(3), 224–274. https://doi.org/10.1145/501978.501980

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105. https://doi.org/10.2307/25148625

Hu, V. C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., & Scarfone, K. (2014). *Guide to attribute based access control (ABAC) definition and considerations (NIST Special Publication 800-162)*. National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-162

International Organization for Standardization. (2018). *Systems and software engineering—Life cycle processes—Requirements engineering (ISO/IEC/IEEE Standard No. 29148:2018)*. https://www.iso.org/standard/72089.html

International Organization for Standardization. (2022a). *Software engineering—Software life cycle processes—Maintenance (ISO/IEC/IEEE Standard No. 14764:2022)*. https://www.iso.org/standard/80710.html

International Organization for Standardization. (2022b). *Software and systems engineering—Software testing—Part 1: General concepts (ISO/IEC/IEEE Standard No. 29119-1:2022)*. https://www.iso.org/standard/81291.html

International Organization for Standardization. (2023). *Systems and software engineering—Systems and software quality requirements and evaluation (SQuaRE)—Product quality model (ISO/IEC Standard No. 25010:2023)*. https://www.iso.org/standard/78176.html

Joint Task Force. (2020). *Security and privacy controls for information systems and organizations (NIST Special Publication 800-53, Revision 5)*. National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-53r5

Kent, K., & Souppaya, M. (2006). *Guide to computer security log management (NIST Special Publication 800-92)*. National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-92

National Cybersecurity Authority. (2024). *Essential cybersecurity controls (ECC-2:2024)*. https://nca.gov.sa/en/regulatory-documents/controls-list/ecc/

OWASP Foundation. (2025). *OWASP Top 10:2025*. https://top10.owasp.org/2025/

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, *24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). *Zero trust architecture (NIST Special Publication 800-207)*. National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-207

Røstad, L., & Edsberg, O. (2006). A study of access control requirements for healthcare systems based on audit trails from access logs. In *2006 22nd Annual Computer Security Applications Conference (ACSAC'06)* (pp. 175–186). IEEE. https://doi.org/10.1109/ACSAC.2006.8

Saudi Data and Artificial Intelligence Authority. (2023). *Guide to the Saudi Personal Data Protection Law for controllers and processors*. https://dgp.sdaia.gov.sa/wps/portal/pdp/knowledgecenter/details/PDPLCP

Souppaya, M., Morello, J., & Scarfone, K. (2017). *Application container security guide (NIST Special Publication 800-190)*. National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-190

Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure Software Development Framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities (NIST Special Publication 800-218)*. National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-218

Suleski, T., & Ahmed, M. (2023). A data taxonomy for adaptive multifactor authentication in the Internet of Health Care Things. *Journal of Medical Internet Research, 25,* Article e44114. https://doi.org/10.2196/44114

Suleski, T., Ahmed, M., Yang, W., & Wang, E. (2023). A review of multi-factor authentication in the Internet of Healthcare Things. *Digital Health, 9,* Article 20552076231177144. https://doi.org/10.1177/20552076231177144

Summers, B. L. (2020). *Effective methods for software engineering*. Auerbach Publications. https://doi.org/10.1201/9781003025665

Temoshok, D., Fenton, J. L., Choong, Y.-Y., Lefkovitz, N., Regenscheid, A., Galluzzo, R., & Richer, J. P. (2025). *Digital identity guidelines: Authentication and authenticator management (NIST Special Publication 800-63B-4)*. National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-63B-4

Vukotich, G. (2023). Healthcare and cybersecurity: Taking a zero trust approach. *Health Services Insights, 16,* Article 11786329231187826. https://doi.org/10.1177/11786329231187826

# APPENDICES

## Appendix A: Test Commands and Evidence Map

The following commands reproduce the Unit 7 and final-submission evidence from the repository root after installing the pinned dependencies in a virtual environment.

```
python -m pytest tests/test_unit7_system_acceptance.py -v
python -m pytest -v > results/unit7_full_test_log.txt
python -m pytest -q
python system_demo.py
python performance_evaluation.py
```

- tests/test_access_control.py: 15 policy-focused unit tests.
- tests/test_system_integration.py: three service–repository integration tests.
- tests/test_unit7_system_acceptance.py: seven system, acceptance, and stability tests (Table 8).
- results/unit7_full_test_log.txt: verbose 25-test execution record (Windows, Python 3.12.15, pytest 9.1.1).
- results/evaluation_results.csv, results/evaluation_environment.json, and results/system_evaluation_chart.png: saved outputs of the final evaluation run (36 labeled cases, latency percentiles, hardware record).
- Figures 5–7: initial failure, corrected focused run, and final regression result.

## Appendix B: Configuration and Deployment Summary

The documented configuration names and local commands are reproduced below; the local SQLite path and synthetic-data setting are demonstration defaults only.

```
EHR_ENVIRONMENT=development
EHR_AUDIT_DB_PATH=data/ehr_audit.db
EHR_LOG_LEVEL=INFO
EHR_AUDIT_SECRET=<inject from an approved secrets manager>
EHR_USE_SYNTHETIC_DATA=true

python -m venv .venv
python -m pip install -r requirements.txt
python -m pytest -q
docker build -t integrated-ehr-access-control:1.2.0 .
docker run --rm --env-file .env -v ehr-audit-data:/app/data \
    integrated-ehr-access-control:1.2.0
```

Production deployment must replace the example secret, use an approved identity provider and managed database, enforce TLS, centralize monitoring, scan and sign the image, and complete governance approval.

## Appendix C: Repository and Release

Repository: https://github.com/Taghreed-Almirabi/Integrated-EHR-Access-Control-Framework

Verified Unit 7 release: https://github.com/Taghreed-Almirabi/Integrated-EHR-Access-Control-Framework/releases/tag/v1.2.0-unit7

Final release: https://github.com/Taghreed-Almirabi/Integrated-EHR-Access-Control-Framework/releases/tag/v1.3.0-final — commit 0bb3017 (0bb301769252c32b67086c8cf34520670baf0268), committed to main on October 24, 2026, after the complete suite passed (25/25).

Project portfolio (GitHub Pages): https://taghreed-almirabi.github.io/capstone-portfolio/

Presentation and demonstration video (ScreenPal, 13 min 35 s): https://go.screenpal.com/watch/cO6u2Vny5oo

Verified Unit 7 merge commit: d2ac7cd (d2ac7cdd4ae9c99217a074affd515bb8a63298a9), merged into main on October 17, 2026.

Earlier milestones: v1.1.0-unit6 (a4fbbfe), v1.0.0 (cc2766a), v0.2.0-tested (5a7e9ab), and v0.1.0-core (76910c7).

## Appendix D: Expert Reviewer Validation Instrument and Completed Forms

Two healthcare IT experts completed the instrument below on October 24, 2026 (Section 4.3.3): each received the public portfolio link, exercised the demonstration scenarios in the browser, rated statements C1–C10 from 1 (strongly disagree) to 5 (strongly agree), answered open questions on strengths, weaknesses, and improvements, stated whether a hospital pilot is recommended, and signed a declaration of independent judgment and synthetic data only. The success threshold was a mean of at least 4.0 with no unresolved critical concern.

The completed and signed forms are reproduced below with the reviewers' permission (Figures D1–D6). Reviewer 1 is the external reviewer; Reviewer 2 is the author's brother, as disclosed in Section 4.3.3.

**Figure D1**

*Reviewer 1 Evaluation Form, Page 1 (Information and Review Steps)*

Appendix D

## Expert Reviewer Evaluation Form

*Designing and Evaluating an Integrated Access Control Framework for Electronic Health Records in a Saudi Government Hospital*

Taghreed Y. Almirabi | MSIT 5910-01 Capstone Project | University of the People | Supervisor: Dr. Stella T. Whyte

Thank you for reviewing this academic prototype. The review takes about 20 minutes. The system uses synthetic users and patients only; no real patient information is involved. Your name and role will appear in the capstone report only with your permission, and your ratings will be reported as aggregated scores.

### Section A. Reviewer information

| | |
|---|---|
| **Full name** | Basem Siddiq Ali |
| **Job title / role** | Senior Healthcare Information Security Consultant |
| **Organization (optional)** | Healthcare IT Security Advisory |
| **Years of experience in healthcare IT or information security** | 12 years |
| **Date of review** | October 24, 2026 |

### Section B. What to review

Please complete the following steps before rating. No installation is needed; everything runs in your web browser.

1. Open the project portfolio: https://taghreed-almirabi.github.io/capstone-portfolio/
2. In the panel “Try an access request”, click each of the four scenario buttons and read the decision, the reason and the audit line that appear below the form.
3. Change at least two settings yourself. For example, untick “On duty” and watch the decision change; tick “Emergency (break-glass)” and type a justification; select “IT systems administrator” and request “View patient record”.
4. Read the section “How a request is decided” and the role matrix.
5. Optional: browse the “Evidence” section (test suite, measured results) and the “Deliverables” section (source code and documentation).

*Note.* Signed evaluation form reproduced with the reviewer's permission; synthetic data only.

**Figure D2**

*Reviewer 1 Evaluation Form, Page 2 (Ratings)*

### Section C. Ratings

Mark one box per statement with an X. Scale: 1 = Strongly disagree, 2 = Disagree, 3 = Neutral, 4 = Agree, 5 = Strongly agree.

| No. | Statement | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| **Security behaviour** | | | | | | |
| C1 | The role permissions follow least privilege and match how a hospital actually works. | | | | | X |
| C2 | The contextual checks (duty period, patient assignment, network, device) are appropriate for a hospital EHR. | | | | | X |
| C3 | Step-up MFA is required in the right situations (record export, account management, off-network access, unmanaged device). | | | | | X |
| C4 | The break-glass emergency path is controlled appropriately (clinical roles only, written justification, 15-minute expiry, flagged for review). | | | | | X |
| C5 | The audit trail gives enough information for accountability and investigation, and tamper detection adds real value. | | | | | X |
| **Usability and clarity** | | | | | | |
| C6 | The decision labels (PERMIT, DENY, STEP_UP_MFA, BREAK_GLASS_PERMIT) are clear and unambiguous. | | | | X | |
| C7 | The reason given for each decision would be understandable to a non-technical clinician or administrator. | | | | | X |
| C8 | The inputs are consistent and the system is easy to learn. | | | | | X |
| C9 | Audit traceability: I can tell who requested what, when, and what was decided. | | | | | X |
| **Overall** | | | | | | |
| C10 | With further development, this framework is applicable in a real Saudi government hospital environment. | | | | X | |

*Note.* Signed evaluation form reproduced with the reviewer's permission; synthetic data only.

**Figure D3**

*Reviewer 1 Evaluation Form, Page 3 (Open Feedback and Declaration)*

## Section D. Open feedback

| D1. Main strengths of the framework (two or three points) |
|---|
| 1. Comprehensive RBAC & Contextual Integration: Excellent implementation of the principle of least privilege combined with dynamic contextual parameters (duty status, patient assignment, device trust, secure network location) tailored specifically to clinical workflows.<br>2. Secure Break-Glass Emergency Mechanism: Well-structured emergency override path balancing urgent patient care necessities with strict accountability through mandatory justification, time-bound access (15-minute expiry), and instant audit flagging.<br>3. Accountability & Integrity: The inclusion of tamper-evident audit logging provides high traceability and satisfies essential healthcare compliance requirements. |

| D2. Weaknesses, risks or gaps you noticed |
|---|
| 1. Integration Complexity with Legacy Systems: Integrating this framework with legacy Electronic Health Record (EHR) platforms and proprietary hospital systems deployed across Saudi hospitals may present technical overhead.<br>2. Latency during Peak Hours: Real-time multi-factor authentication (MFA) step-up triggers and context verification could introduce latency during emergency situations if network infrastructure experiences bottlenecks. |

| D3. Suggested improvements before a hospital pilot |
|---|
| 1. Conduct rigorous concurrency and latency load testing to ensure real-time responsiveness during high-pressure emergency department shifts.<br>2. Align the authentication module with national Saudi digital identity standards (e.g., integration with NAFATH / unified governmental single sign-on frameworks).<br>3. Build an advanced administrative dashboard for security officers to streamline post-incident reviews of break-glass activations. |

D4. Would you recommend piloting this framework in a hospital setting after further development? Yes / [X] Yes, with the changes above / No

## Section E. Declaration

I confirm that this evaluation reflects my independent professional judgement, that I reviewed the prototype described in Section B, and that no real patient data was shown to me.

| Reviewer signature | Date |
|---|---|
| Basem Siddiq Ali | October 24, 2026 |

*Please return the completed form to the student by email. Thank you for your time and expertise.*

*Note.* Signed evaluation form reproduced with the reviewer's permission; synthetic data only.

**Figure D4**

*Reviewer 2 Evaluation Form, Page 1 (Information and Review Steps)*

Appendix D

## Expert Reviewer Evaluation Form

*Designing and Evaluating an Integrated Access Control Framework for Electronic Health Records in a Saudi Government Hospital*

Taghreed Y. Almirabi | MSIT 5910-01 Capstone Project | University of the People | Supervisor: Dr. Stella T. Whyte

Thank you for reviewing this academic prototype. The review takes about 20 minutes. The system uses synthetic users and patients only; no real patient information is involved. Your name and role will appear in the capstone report only with your permission, and your ratings will be reported as aggregated scores.

### Section A. Reviewer information

| | |
|---|---|
| **Full name** | Abdulrahim Yousif Almirabi |
| **Job title / role** | Senior Health Informatics & IT Operations Manager |
| **Organization (optional)** | Saudi Health Digital Transformation Directorate |
| **Years of experience in healthcare IT or information security** | 14 years |
| **Date of review** | October 24, 2026 |

### Section B. What to review

Please complete the following steps before rating. No installation is needed; everything runs in your web browser.

1. Open the project portfolio: https://taghreed-almirabi.github.io/capstone-portfolio/
2. In the panel “Try an access request”, click each of the four scenario buttons and read the decision, the reason and the audit line that appear below the form.
3. Change at least two settings yourself. For example, untick “On duty” and watch the decision change; tick “Emergency (break-glass)” and type a justification; select “IT systems administrator” and request “View patient record”.
4. Read the section “How a request is decided” and the role matrix.
5. Optional: browse the “Evidence” section (test suite, measured results) and the “Deliverables” section (source code and documentation).

*Note.* Signed evaluation form reproduced with the reviewer's permission; synthetic data only.

**Figure D5**

*Reviewer 2 Evaluation Form, Page 2 (Ratings)*

### Section C. Ratings

Mark one box per statement with an X. Scale: 1 = Strongly disagree, 2 = Disagree, 3 = Neutral, 4 = Agree, 5 = Strongly agree.

| No. | Statement | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| **Security behaviour** | | | | | | |
| C1 | The role permissions follow least privilege and match how a hospital actually works. | | | | | X |
| C2 | The contextual checks (duty period, patient assignment, network, device) are appropriate for a hospital EHR. | | | | | X |
| C3 | Step-up MFA is required in the right situations (record export, account management, off-network access, unmanaged device). | | | | | X |
| C4 | The break-glass emergency path is controlled appropriately (clinical roles only, written justification, 15-minute expiry, flagged for review). | | | | | X |
| C5 | The audit trail gives enough information for accountability and investigation, and tamper detection adds real value. | | | | | X |
| **Usability and clarity** | | | | | | |
| C6 | The decision labels (PERMIT, DENY, STEP_UP_MFA, BREAK_GLASS_PERMIT) are clear and unambiguous. | | | | | X |
| C7 | The reason given for each decision would be understandable to a non-technical clinician or administrator. | | | | | X |
| C8 | The inputs are consistent and the system is easy to learn. | | | | X | |
| C9 | Audit traceability: I can tell who requested what, when, and what was decided. | | | | | X |
| **Overall** | | | | | | |
| C10 | With further development, this framework is applicable in a real Saudi government hospital environment. | | | | | X |

*Note.* Signed evaluation form reproduced with the reviewer's permission; synthetic data only.

**Figure D6**

*Reviewer 2 Evaluation Form, Page 3 (Open Feedback and Declaration)*

## Section D. Open feedback

| D1. Main strengths of the framework (two or three points) |
|---|
| 1. User-Centric Clinical Workflow Alignment: The integration of context-aware policies (role, duty status, patient assignment) seamlessly bridges security constraints with fast-paced hospital operations.<br>2. Robust Emergency Protocols: The break-glass feature is meticulously designed with mandatory justification and automatic 15-minute expiration, solving a critical healthcare security dilemma.<br>3. Audit Traceability & Compliance: The tamper-evident logging mechanism ensures high accountability, aligning well with regional healthcare regulatory standards (such as SFDA and CBAHI guidelines). |

| D2. Weaknesses, risks or gaps you noticed |
|---|
| 1. User Onboarding and Training Overhead: Clinicians might require initial orientation to understand step-up MFA prompts and context validation rules during hectic shifts.<br>2. Centralized Directory Dependency: High reliance on real-time directory services for context checks could introduce minor bottlenecks if redundancy isn't fully optimized. |

| D3. Suggested improvements before a hospital pilot |
|---|
| 1. Implement automated caching mechanisms for offline or degraded network scenarios to maintain clinical continuity.<br>2. Provide role-specific micro-training modules directly within the user interface for first-time users.<br>3. Expand the audit export feature to support standard SIEM connectors (e.g., Splunk, ELK stack) for hospital SOC integration. |

D4. Would you recommend piloting this framework in a hospital setting after further development? Yes / [X] Yes, with the changes above / No

## Section E. Declaration

I confirm that this evaluation reflects my independent professional judgement, that I reviewed the prototype described in Section B, and that no real patient data was shown to me.

| Reviewer signature | Date |
|---|---|
| Abdulrahim Yousif Almirabi  | October 24, 2026 |

*Please return the completed form to the student by email. Thank you for your time and expertise.*

*Note.* Signed evaluation form reproduced with the reviewer's permission; synthetic data only.

## Appendix E: Code Excerpts

Service orchestration (src/ehr_access_service.py), reproduced from the released commit:

```
def process_request(self, request: AccessRequest) -> DecisionResult:
    """Evaluate one request and store its audit event."""
    result = self.engine.evaluate(request)
    event = self._find_audit_event(result.audit_event_id)
    self.repository.save_event(event)
    return result
Audit signing and verification (src/access_control_engine.py):
def _sign(self, payload: dict[str, object]) -> str:
    encoded = json.dumps(
        payload, sort_keys=True, separators=(",", ":")
    ).encode()
    return hmac.new(
        self.audit_secret, encoded, hashlib.sha256
    ).hexdigest()

def verify_audit_event(self, event: AuditEvent) -> bool:
    expected = self._sign({...event fields except checksum...})
    return hmac.compare_digest(event.checksum, expected)
```